

Supplementary Materials: All Emulators are Wrong, Many are Useful, and Some are More Useful Than Others

A Reproducible Comparison of Computer Model Surrogates

Kellin N. Rumsey

Devin Francom

Graham C. Gibson

Reid Morris

Table of contents

SM 1 R Scripts	2
SM 2 Summary of Emulator Performance	2
SM 2.1 Overall Performance Summary	2
SM 2.2 Best Emulators for Each Function	5
SM 2.2.1 Test Functions	5
SM 2.2.2 Real Datasets	7
SM 3 Reproducible Simulation Studies	9
SM 3.1 Seed Transformations	9
SM 3.2 The Fallback Model	9
SM 3.3 Simulation Study Output	10
SM 3.4 Datasets	10
SM 3.5 Test Functions (and Characteristics)	12
SM 4 The Rashomon Effect and Ensembling	14
SM 5 Additional Figures and Analysis	15
SM 5.1 Fraction of Variance Unexplained	16
SM 5.1.1 FVU Rankplots	16
SM 5.1.2 FVU Performance Profiles	21
SM 5.1.3 FVU Heatmaps	26
SM 5.1.4 FVU Boxplots	33
SM 5.2 Continuous Ranked Probability Scores	34
SM 5.2.1 CRPS Rankplots (data only)	34
SM 5.2.2 CRPS Performance Profiles (testfuncs only)	36
SM 5.3 Computation Time	39
SM 5.3.1 Time Performance Profiles	39
SM 5.3.2 Time Paretoplots	44
SM 5.4 Other Metrics	45
References	47

SM 1 R Scripts

Most of the figures in the main paper can be recreated by running the script https://github.com/knrumsey/duqling_results/R/duqling/duqling_paper_main.R. In addition, the exact implementation for each of the 32 emulation methods can be found in the `emulators.R` file at https://github.com/knrumsey/duqling_results/R/duqling/emulators.R. Alternatively the implementation functions can be found using the `duqling` package.

```
# Install and load duqling
# devtools::install_github("knrumsey/duqling")
library(duqling)
funcs <- get_emulator_functions("bass")
fit_bass <- funcs$fit_func
pred_bass <- funcs$pred_func
```

SM 2 Summary of Emulator Performance

The package makes it easy to conduct additional analyses. We start by loading the data into the global environment. Then we use the `process_sim_study()` function to get the data frames ready for the `duqling` functions, which expect an object of class `duq_sim_study`.

```
# Sim study on the 60 test functions
data("sim_study_testfuncs")
duq_testfuncs <- process_sim_study(sim_study_testfuncs)

# Sim study on the 40 test functions
data("sim_study_realdata")
duq_realdata <- process_sim_study(sim_study_realdata)
```

SM 2.1 Overall Performance Summary

For each of the 32 surrogate modeling methods, we look at

- Average runtime
- CRPS: winrate, top10rate, and average soft-relative scores
- FVU: winrate, top10rate, and average soft-relative scores

The average soft-relative scores give us a rough measure of practical performance. For emulator i and simulation scenario j we define

$$t_{i,j} = \frac{\text{metric}_{i,j} + \epsilon}{\min_i \{\text{metric}_{i,j}\} + \epsilon},$$

and the average soft-relative score is simply

$$\text{Soft-Rel-Score}_i = J^{-1} \sum_{j=1}^J \min(t_{i,j}, U)$$

where ϵ represents a point at which the metric should be considered practically 0 and U is a truncation point for robustness.

These metrics can be summarized in `duqling` with the code.

```
duq_total <- join_sim_study(duq_testfuncs, duq_realdata)
table <- summarize_sim_study(duq_total,
  summarize="time", summary_fun = function(val) c(mean=mean(val)),
  win_rate = c("CRPS", "FVU"),
  topK_rate = c("CRPS", "FVU"), K=10,
  soft_rel = c("CRPS", "FVU"),
  epsilon=c(0.001, 0.001), upper_bound=c(1e4, 1e4))
```

For readability, we have broken this into two tables for CRPS and FVU separately, given below.

Method	Time (mean)	CRPS (win rate)	CRPS (top 10%)	CRPS (soft relative)
alcgp	14.216	5.882e-04	0.095	28.699
apce	67.243	0.111	0.384	15.939
bart	21.753	0.032	0.297	14.841
baseline	0.133	0.019	0.031	155.956
bass	21.855	0.041	0.673	5.239
bcart	27.684	0.000e+00	0.025	67.270
bcmgp	54.824	0.002	0.211	25.987
blasso	0.249	0.003	0.025	113.389
blm	0.102	0.029	0.069	88.958
bnn	4134.652	0.000e+00	0.069	255.358
bootrf	495.688	0.000e+00	0.093	41.584
bppr	27.401	0.039	0.766	9.086
btreelm	33.622	0.002	0.171	22.037
confrf	10.280	0.012	0.186	34.865
deepgp	4868.729	0.091	0.526	16.877
fitcgp	17.204	0.013	0.308	35.433
gbc	67.393	0.085	0.454	10.204
gp	211.286	0.043	0.799	7.069
gpytorch	151.071	0.003	0.229	481.235
hetgp	429.467	0.044	0.768	6.784
lagp	132.919	0.004	0.281	19.634
mpgp	131.328	0.000e+00	0.108	64.867
ngboost	188.036	8.824e-04	0.078	80.199
qbass	306.078	0.004	0.374	308.943
rffgp	5.468	0.006	0.134	85.564
rgasp	765.851	0.322	0.955	4.211
rvm	195.968	0.007	0.229	63.031
spce	709.144	0.017	0.172	144.580
svecgp	80.879	0.125	0.799	29.632
svigp	60.420	8.824e-04	0.336	12.096
tbass	373.883	0.031	0.624	9.252
treegp	40.922	0.021	0.334	45.345

```

# ---- FVU TABLE ----
table_fvu <- summarize_sim_study(
  duq_total,
  summarize = "time", summary_fun = function(val) c(mean = mean(val)),
  win_rate = "FVU",
  topK_rate = "FVU", K = 10,
  soft_rel = "FVU", epsilon = 0.001, upper_bound=1e4
)

# Convert to numeric
for(j in 2:ncol(table_fvu)){
  table_fvu[,j] <- as.numeric(table_fvu[,j])
}

# Format numeric columns
table_fvu$time_mean <- fmt(table_fvu$time_mean)
table_fvu$FVU_winrate <- fmt(table_fvu$FVU_winrate)
table_fvu$FVU_top10rate <- fmt(table_fvu$FVU_top10rate)
table_fvu$FVU_softrel <- fmt(table_fvu$FVU_softrel)

# Rename columns
colnames(table_fvu) <- c(
  "Method",
  "Time (mean)",
  "FVU (win rate)",
  "FVU (top 10%)",
  "FVU (soft relative)"
)

knitr::kable(table_fvu, align = "lcccc")

```

Method	Time (mean)	FVU (win rate)	FVU (top 10%)	FVU (soft relative)
alcgp	14.216	2.941e-04	0.084	116.669
apce	67.243	0.118	0.394	23.935
bart	21.753	0.036	0.262	10.632
baseline	0.133	0.018	0.034	441.186
bass	21.855	0.045	0.639	8.191
bcart	27.684	0.000e+00	0.021	117.843
bcmgp	54.824	0.002	0.230	52.891
blasso	0.249	0.004	0.029	248.636
blm	0.102	0.028	0.065	182.037
bnn	4134.652	2.941e-04	0.061	471.030
bootrf	495.688	0.002	0.166	46.966
bppr	27.401	0.037	0.755	7.579
btreelm	33.622	0.002	0.137	34.050
confrf	10.280	0.011	0.181	40.661
deepgp	4868.729	0.074	0.438	28.761

Method	Time (mean)	FVU (win rate)	FVU (top 10%)	FVU (soft relative)
fitcgp	17.204	0.007	0.279	59.866
gbc	67.393	0.057	0.422	14.898
gp	211.286	0.048	0.787	8.369
gpytorch	151.071	0.016	0.414	811.459
hetgp	429.467	0.039	0.697	8.880
lagp	132.919	0.004	0.265	65.591
mpgp	131.328	5.882e-04	0.099	33.455
ngboost	188.036	0.002	0.063	129.674
qbass	306.078	0.007	0.409	316.246
rffgp	5.468	0.003	0.124	158.576
rgasp	765.851	0.349	0.939	2.677
rvn	195.968	0.008	0.177	113.715
spce	709.144	0.021	0.227	380.335
svecgp	80.879	0.131	0.769	53.861
svigp	60.420	0.008	0.525	22.005
tbass	373.883	0.023	0.562	13.940
treegp	40.922	0.011	0.310	145.445

The reader may be interested in recreating these tables after filtering to a specific set of simulation scenarios, but we omit this here for brevity.

SM 2.2 Best Emulators for Each Function

To summarize where each emulator performs particularly well, we identify methods that attain the single best CRPS in at least one case. We encourage the reader to filter the simulation study objects before running this code, which we don't do here for the sake of brevity.

SM 2.2.1 Test Functions

```
# Ensure CRPS ranks exist
duq_testfuncs_ranked <- rank_sim_study(duq_testfuncs, metric = "CRPS")

df <- duq_testfuncs_ranked$df

# Keep only winners
best_df <- df[df$CRPS_rank == 1, , drop = FALSE]

# Keep only scenarios with a unique winner
scenario_counts <- ave(best_df$method, best_df$sim_scenario, FUN = length)
best_df <- best_df[scenario_counts == 1, , drop = FALSE]

# Build table: one row per function
ids <- sort(unique(best_df$id))

methods_best <- sapply(ids, function(z) {
```

```

paste(sort(unique(best_df$method[best_df$id == z])), collapse = ", ")
})

n_methods_best <- sapply(ids, function(z) {
  length(unique(best_df$method[best_df$id == z]))
})

total_wins <- sapply(ids, function(z) {
  sum(best_df$id == z)
})

best_table_testfuncs <- data.frame(
  id = ids,
  n_methods_best = n_methods_best,
  methods_best = methods_best,
  stringsAsFactors = FALSE
)

rownames(best_table_testfuncs) <- NULL
colnames(best_table_testfuncs) <- c("Function", "# of Best", "Best Emulators")

knitr::kable(best_table_testfuncs, align = c("l", "c", "l"))

```

Function	# of Best	Best Emulators
banana	2	apce, rffgp
borehole	5	bass, bppr, fitcgp, rgasp, svecgp
cantilever_D	3	deepgp, gbc, svecgp
cantilever_S	3	deepgp, gbc, svecgp
circuit	4	bppr, rffgp, rgasp, svecgp
const_fn	9	baseline, bass, bcmgp, blasso, blm, bppr, rffgp, rgasp, svecgp
const_fn15	10	alcgp, apce, baseline, bass, bcmgp, bppr, deepgp, gp, rffgp, rgasp
crater	3	gbc, rgasp, svecgp
cube3	4	bart, confrf, deepgp, gbc
cube3_rotate	2	deepgp, gbc
cube5	4	bart, bass, bppr, gbc
detpep_curve	6	apce, bass, bppr, fitcgp, rffgp, rgasp
detpep8	4	bass, bppr, rgasp, svecgp
dms_additive	6	apce, bass, bppr, rgasp, svecgp, treegp
dms_complicated	7	bcmgp, deepgp, fitcgp, gp, hetgp, lagp, svecgp
dms_harmonic	2	apce, hetgp
dms_radial	1	apce
dms_simple	4	apce, bass, fitcgp, rffgp
ebola	3	bppr, rgasp, svecgp
forrester1	9	apce, bass, bppr, fitcgp, gp, rffgp, rgasp, rvm, spce
foursquare	8	bart, bass, bppr, btrelm, deepgp, gbc, tbass, treegp
friedman	5	apce, bass, bppr, rgasp, svecgp

Function	# of Best	Best Emulators
friedman10	4	apce, bppr, rgasp, svecgp
friedman20	3	bppr, rgasp, svecgp
Gfunction	6	bass, gbc, gp, rgasp, tbass, treegp
Gfunction18	6	deepgp, gbc, qbass, rgasp, spce, tbass
Gfunction40	4	bppr, qbass, spce, tbass
Gfunction6	5	bass, bppr, gbc, rgasp, tbass
grlee1	4	gp, hetgp, rgasp, treegp
grlee6	5	bass, qbass, rgasp, svecgp, tbass
higdon1	8	bass, btrelm, deepgp, qbass, rgasp, svecgp, tbass, treegp
ignition	2	deepgp, gbc
ishigami	4	apce, hetgp, rgasp, svecgp
lim_non_polynomial	7	bppr, fitcgp, gp, hetgp, rffgp, rgasp, svecgp
lim_polynomial	7	apce, bppr, fitcgp, gp, rffgp, rgasp, rvm
linear21	5	blasso, blm, fitcgp, rffgp, rgasp
linear21_s1	5	apce, bass, blm, bppr, rgasp
linear21_s10	9	apce, bass, blm, bppr, btrelm, fitcgp, rffgp, rgasp, tbass
moon31	3	gbc, rgasp, svecgp
multivalley	5	bart, bass, rgasp, tbass, treegp
onehundred	6	bass, gp, hetgp, rvm, spce, tbass
oo15	2	rgasp, spce
park4	9	apce, bass, fitcgp, gp, hetgp, qbass, rffgp, rgasp, svecgp
permdb	5	confrf, deepgp, ngboost, rvm, tbass
piston	2	rgasp, svecgp
pollutant_uni	5	apce, bass, bppr, rgasp, svecgp
quad4	4	apce, fitcgp, rffgp, rgasp
rabbits	8	bass, bppr, fitcgp, gp, hetgp, rffgp, rgasp, svecgp
ripples	6	deepgp, gp, gpytorch, hetgp, rgasp, svecgp
robot	4	gp, hetgp, rgasp, svecgp
sharkfin	2	bart, tbass
short_column	3	bppr, rgasp, svecgp
squiggle	3	deepgp, gbc, svecgp
star2	5	confrf, deepgp, gbc, rgasp, svecgp
steel_column	2	rgasp, svecgp
sulfur	7	deepgp, gbc, gp, hetgp, lagp, rvm, svigp
turb12	4	gbc, rgasp, svecgp, svigp
twin_galaxies	7	deepgp, fitcgp, gbc, gp, hetgp, rgasp, svecgp
welch20	3	bppr, rgasp, tbass
wingweight	3	bppr, fitcgp, rgasp

SM 2.2.2 Real Datasets

Though not shown, the same code is used as above, with `duq_realdata` in place of `duq_testfuncs`.

Function	# of Best	Best Emulators
acme_climate	2	gp, hetgp
diablo_canyon_plume	1	svecgp
diffusion_1D	1	rgasp
diffusion_2D	1	rgasp
discoflux_flyer	2	gbc, hetgp
e3sm_mcar	2	deepgp, svecgp
e3sm_mnar	2	deepgp, svecgp
fair_climate_ssp1-2.6_year2200	2	hetgp, rgasp
fair_climate_ssp2-4.5_year2200	2	rgasp, svecgp
fair_climate_ssp3-7.0_year2200	2	rgasp, svecgp
flyer_plate104	4	bart, gbc, rgasp, tbass
flyerPTW1	5	bart, gbc, hetgp, rgasp, tbass
flyerPTW2	3	bppr, rgasp, svecgp
icf1	2	bppr, tbass
icf2	2	apce, gp
jc	1	rgasp
nuclear_data	1	bppr
okc_plume	1	gbc
pbx9501_gold	7	bass, bppr, gbc, hetgp, rgasp, svecgp, tbass
pbx9501_nickel	4	fitcgp, gpytorch, rffgp, rgasp
pbx9501_ss304	4	fitcgp, rffgp, rgasp, svecgp
pbx9501_uranium	5	deepgp, gbc, hetgp, rgasp, svecgp
plate_deformation	6	bass, bppr, deepgp, rgasp, svecgp, tbass
ptw1	5	gbc, gp, hetgp, rgasp, svecgp
ptw2	2	hetgp, rgasp
SLOSH_high	2	gbc, hetgp
SLOSH_low	1	gbc
SLOSH_mid	2	gbc, hetgp
spectral1	2	gp, hetgp
spectra2	2	gp, hetgp
stochastic_sir	1	gbc
strontium_plume_p104	1	svecgp
strontium_plume_p4b	5	confrf, hetgp, qbass, svecgp, tbass
taylor_cylinder1	2	bass, gbc
taylor_foot	2	gbc, rgasp
taylor_length	1	rgasp
wind_speed	5	gp, gpytorch, rgasp, svecgp, tbass
Z_machine_max_vel_all	1	svecgp
Z_machine_max_vel1	2	rgasp, svecgp
Z_machine_max_vel2	1	svecgp

SM 3 Reproducible Simulation Studies

SM 3.1 Seed Transformations

One of the primary design principles of `duqling::run_sim_study()` is reproducibility. For any combination of (i) test function, (ii) n_{train} , (iii) design type, (iv) noise to signal ratio, and (v) replication number, the generated training data (inputs and response) are identical. This is true, even when using slightly different function calls. For example,

```
results1 <- run_sim_study(fit_func, pred_func,
                          fnames=c("borehole", "ishigami", "banana"),
                          NSR=c(0, 0.1),
                          design_type="LHS",
                          replications=10)

results2 <- run_sim_study(fit_func, pred_func,
                          fnames=c("ishigami", "borehole"),
                          NSR=0,
                          design_type="LHS",
                          replications=c(1, 7, 15))
```

the rows of `results1` and `results2` corresponding to the same simulation scenario will have identical results (with minor exceptions such as timing data) even though they were run in a different order internally.

To accomplish this, we use a rolling polynomial hash (also called Rabin fingerprinting) to generate a unique seed for each simulation scenario,

$$\text{seed} = \left(\sum_{j=0}^6 B^j s_j \right) \bmod M \quad (1)$$

for prime numbers $B = 101$ and $M = 100030001$, where the set $\{s_0, \dots, s_j\}$ describes the simulation scenario. This can be computed safely using Horner's rule ([1]). The s_j values are computed as follows: s_0 is the user-defined seed with default 42 (not usually intended to be modified), $s_1 = n_{\text{train}}$, s_2 is the replication index. To obtain s_3 , we hash each function name to a unique integer (with high probability) again using the Rabin fingerprint (with $B = 31$) after mapping each character to a unique integer. The pair (s_4, s_5) encodes the noise to signal ratio (NSR) with at least four decimal places of precision as follows:

$$s_4 = \begin{cases} 99999, & NSR = 0 \\ \lceil -10^4 \times \log(NSR) \rceil, & NSR < 1 \\ 0, & NSR \geq 1 \end{cases}, \quad s_5 = \begin{cases} 0, & NSR < 1 \\ \lfloor 10^4 \times NSR \rfloor, & NSR \geq 1 \end{cases}. \quad (2)$$

Finally, $s_6 \in \{1, 2, 3\}$ denotes the design type: LHS, Grid, and Random, respectively.

SM 3.2 The Fallback Model

Even the most robust emulator implementations can occasionally fail, particularly when faced with such a diverse and challenging set of test functions and datasets. To ensure that simulation studies proceed smoothly and results remain well-defined, `duqling::run_sim_study()` incorporates

a fallback strategy. If one of the emulator methods fails during fitting or prediction, it is temporarily replaced with a baseline “null model” that predicts by fitting a Gaussian distribution to the marginal of the response variable from the training data (ignoring the inputs). Specifically, the predictive distribution for each test point is a Student’s t distribution with $n - 1$ degrees of freedom, mean $\bar{y}$, and variance $S_y^2(1 + 1/n)(n - 1)/(n - 3)$, where n , $\bar{y}$, and S_y^2 are the training set size, mean, and sample variance, respectively. When a failure occurs, it is logged in the `failure_type` column of the returned data-frame one of the following values: “none”, “fit”, or “pred”.

SM 3.3 Simulation Study Output

The `run_sim_study()` function returns an R data frame in which each row corresponds to the outcome of a single simulation scenario, replication, and emulator method. The returned columns include:

- **method**: Short name of the fitted emulator; if no name is supplied, a unique label (`method<indx>`) is generated automatically.
- **fname**: The test function name (from `duqling::quack()`).
- **input_dim**: Input dimension of the test function.
- **n**: Number of training points used.
- **NSR**: Noise-to-signal ratio applied in data generation.
- **design_type**: Experimental design type ("LHS", "grid", or "random").
- **rep**: Replication index.
- **t_fit**, **t_pred**, **t_tot**: Elapsed wall-clock time (seconds) for fitting, prediction, and total runtime, respectively.
- **failure_type**: Categorical indicator for whether the method failed at fitting, prediction, or "none" if successful.
- **RMSE**: Root mean squared error on the test set.
- **FVU**: Fraction of variance unexplained, computed as RMSE^2 divided by the total variance.
- **COVER**, **MIS**: Coverage rates and mean interval scores for each specified confidence level.
- **IAE_alpha**: Integrated absolute error of empirical coverage across confidence levels, summarizing calibration of predictive intervals.
- **CRPS**: Mean continuous ranked probability score on the test set, along with summary statistics (`CRPS_min`, `CRPS_Q1`, `CRPS_med`, `CRPS_Q3`, `CRPS_max`) describing the distribution of CRPS across test points.

Additional columns may be present depending on the metrics and simulation options selected by the user (notably mean interval lengths and negatively oriented mean interval scores). The full results for the simulation studies shown in Section 4 are included in the associated GitHub repository.

SM 3.4 Datasets

A full list of the real datasets used in this comparison is given in the table below. We are hopeful that all of these will be available in `duqling` soon. You can check which emulation datasets are available in `duqling` via the command:

```
data_quack(raw=FALSE, response_type="uni")
```

	dname	input_dim	input_cat_dim	n	response
1	e3sm	2	0	48602	univariate
2	e3sm_mcar	2	0	10000	univariate

3	e3sm_mnar	2	0	9122	univariate
4	stochastic_sir	4	0	2000	univariate
5	SLOSH_high	5	0	4000	univariate
6	SLOSH_low	5	0	4000	univariate
7	SLOSH_mid	5	0	4000	univariate
8	pbx9501_gold	6	0	500	univariate
9	pbx9501_nickel	6	0	500	univariate
10	pbx9501_ss304	6	0	500	univariate
11	pbx9501_uranium	6	0	500	univariate
12	Z_machine_max_vel1	6	0	5000	univariate
13	Z_machine_max_vel2	6	0	5000	univariate
14	flyer_plate104	11	0	1000	univariate
15	strontium_plume_p104	20	0	300	univariate
16	strontium_plume_p4b	20	0	300	univariate
17	Z_machine_max_vel_all	30	0	5000	univariate
18	fair_climate_ssp1-2.6_year2200	45	0	1001	univariate
19	fair_climate_ssp2-4.5_year2200	45	0	1001	univariate
20	fair_climate_ssp3-7.0_year2200	45	0	1001	univariate
21	fair_climate_ssp1-2.6	46	0	56056	univariate
22	fair_climate_ssp2-4.5	46	0	56056	univariate
23	fair_climate_ssp3-7.0	46	0	56056	univariate

A particular dataset can be read into R (internet connection is required) with the command:

```
emu_data <- get_emulation_data("pbx9501_gold")
X <- emu_data$X
y <- emu_data$y
```

The full list of datasets is given below.

Dataset	n	p	In duqling	Reference
plate_deformation	138	7	No	[2]
wind_speed	200	17	No	[3]
strontium_plume_p4b	300	20	Yes	[4]
strontium_plume_p104	300	20	Yes	[4]
spectral1	500	4	No	[5]
spectra2	500	4	No	[5]
pbx9501_gold	500	6	Yes	[6]
pbx9501_ss304	500	6	Yes	[6]
pbx9501_nickel	500	6	Yes	[6]
pbx9501_uranium	500	6	Yes	[6]
ptw1	500	10	No	[7] [8]
discoflux_flyer	914	6	No	[9]
taylor_foot	962	11	No	[10]
taylor_length	962	11	No	[10]
jc	1000	5	No	[7] [11]
flyerPTW1	1000	11	No	[10]
flyerPTW2	1000	11	No	[10]

Dataset	n	p	In duqling	Reference
flyer_plate104	1000	11	Yes	[12]
diffusion_1D	1000	62	No	[13]
fair_climate_ssp1-2.6_year2200	1001	45	Yes	[14]
fair_climate_ssp2-4.5_year2200	1001	45	Yes	[14]
fair_climate_ssp3-7.0_year2200	1001	45	Yes	[14]
taylor_cylinder1	1438	8	No	[15]
stochastic_sir	2000	4	Yes	[16]
diffusion_2D	2000	53	No	[13]
acme_climate	2980	66	No	[17]
ptw2	3701	13	No	[7] [8]
SLOSH_low	4000	5	Yes	[18]
SLOSH_mid	4000	5	Yes	[18]
SLOSH_high	4000	5	Yes	[18]
Z_machine_max_vel1	5000	6	Yes	[19]
Z_machine_max_vel2	5000	6	Yes	[19]
Z_machine_max_vel_all	5000	30	Yes	[19]
e3sm_mnar	9122	2	Yes	[20]
nuclear_data	9307	178	No	[21]
e3sm_mcar	10000	2	Yes	[20]
icf1	10000	5	No	[22]
icf2	10000	5	No	[22]
diablo_canyon_plume	18000	13	No	[23]
okc_plume	22229	5	No	[24]

SM 3.5 Test Functions (and Characteristics)

The `duqling` library of test functions is ever growing, but the list of the 60 test functions studied in this paper can be found as

```
get_paper_funcs()
```

For these 60 functions, we computed several characteristics of interest. Future work could try to link these characteristics (or others not listed here) to emulator performance. Determining, in advance, which surrogate modeling technique is likely to succeed on a given computer model would be of great use to practitioners. The R squared quantity refers to the R^2 statistic from a linear model fit to the input-output pairs of the computer model. For the other methods, gradients were estimated using automatic differentiation when possible. When needed, the gradient is approximated based on a BASS surrogate fit to the computer model. The AS Participation metric refers to the quantity

$$(\sum_{i=1}^p \lambda_i)^2 / \sum_{i=1}^p \lambda_i^2,$$

where λ_i is the i th eigenvalue of the active subspace matrix $C = E(\nabla f(x) \nabla f(x)^T)$.

fname	input_dim	Rsquared	MoransI	Sobolev	Participation
banana	2	0.035	0.955	11.017	1.232

fname	input_dim	Rsquared	MoransI	Sobolev	Participation
borehole	8	0.947	0.791	3.729	1.020
cantilever_D	6	0.501	0.701	5.993	1.159
cantilever_S	6	0.699	0.827	4.495	1.481
circuit	6	0.973	0.855	3.869	1.104
const_fn	1	0.00003	0.020	1.000	
const_fn15	15	0.001	0.001	1.000	1.271
crater	7	0.647	0.758	5.090	1.621
cube3	3	0.060	0.795	31.274	1.332
cube3_rotate	3	0.198	0.797	10.383	2.355
cube5	5	0.622	0.785	19.961	1.323
detpep_curve	3	0.795	0.956	4.611	1.165
detpep8	8	0.802	0.752	4.609	1.178
dms_additive	2	0.058	0.984	9.895	1.918
dms_complicated	2	0.192	0.962	9.831	1.941
dms_harmonic	2	0.030	0.932	13.037	1.735
dms_radial	2	0.0001	0.983	7.092	1.998
dms_simple	2	0.189	0.948	4.768	1.966
ebola	8	0.856	0.748	4.794	1.533
forrester1	1	0.043	0.982	11.682	1
foursquare	2	0.471	0.931	24.490	1.206
friedman	5	0.770	0.877	4.856	2.717
friedman10	10	0.742	0.674	5.064	2.666
friedman20	20	0.753	0.675	4.961	2.674
Gfunction	3	0.0001	0.883	1.000	2.803
Gfunction18	18	0.0001	0.139	1.000	4.144
Gfunction40	40	0.001	0.021	1.000	3.938
Gfunction6	6	0.002	0.562	1.000	2.114
grlee1	1	0.604	0.987	16.676	1
grlee6	6	0.302	0.433	40.708	1.004
higdon1	1	0.520	0.996	34.114	
ignition	10	0.815	0.729	3.937	2.695
ishigami	3	0.199	0.841	11.257	2.380
lim_non_polynomial	2	0.643	0.974	5.532	1.814
lim_polynomial	2	0.594	0.958	6.212	1.764
linear21	21	1	0.569	3.620	1
linear21_s1	21	1	0.572	3.605	1
linear21_s10	21	1	0.594	3.591	1
moon31	31	0.926	0.463	3.764	1.105
multivalley	2	0.091	0.701	72.760	1.488
onehundred	100	0.884	0.279	4.361	2.519
oo15	15	0.258	0.574	1.000	7.538
park4	4	0.955	0.919	3.655	1.075
permdb	16	0.0004	0.101	46.941	9.681
piston	7	0.914	0.819	4.093	1.270
pollutant_uni	4	0.827	0.886	5.231	1.301
quad4	4	0.938	0.920	3.826	1.226

fname	input_dim	Rsquared	MoransI	Sobolev	Participation
rabbits	3	0.927	0.954	4.710	1.072
ripples	2	0.003	0.807	40.944	1.376
robot	8	0.174	0.410	8.614	2.428
sharkfin	3	0.246	0.753	41.910	2.246
short_column	5	0.769	0.824	4.730	1.066
squiggle	2	0.015	0.868	25.773	1.855
star2	2	0.204	0.803	22.482	1.596
steel_column	9	0.968	0.762	3.813	1.139
sulfur	9	0.549	0.698	4.473	1.178
turb12	12	0.335	0.463	7.659	1.758
twin_galaxies	2	0.442	0.962	9.100	1.775
welch20	20	0.598	0.598	6.375	2.097
wingweight	10	0.984	0.758	3.652	1.061

SM 4 The Rashomon Effect and Ensembling

We have already seen that searching for a single emulator that performs best across all functions is a fruitless endeavor—but perhaps we should not even be focused on finding the best emulator for a single setting. “*Amazing things come from having many good models*” is the sentiment behind the Rashomon effect, an idea that has gained increasing attention in recent years ([25]; [26]; [27]; [28]). The term was originally coined by Leo Breiman to describe the phenomenon in which multiple distinct models explain the data equally well, yet may lead to different conclusions ([29]). While the Rashomon effect is often discussed in the context of noisy or over-parameterized models, we believe it also holds valuable lessons for uncertainty quantification.

Suppose that for a given dataset, we retain all emulators with a CRPS below 0.25 (see Figure below). If we then perform a sensitivity analysis using each of these similarly accurate emulators and they all agree, this serves as a strong sanity check—not just for the conclusions, but also for the models themselves. On the other hand, if the emulators disagree, this immediately signals a need for deeper inspection. For example, imagine that one emulator identifies $\mathbf{x}_1$ as the most important input, $\mathbf{x}_2$ as moderately important, and $\mathbf{x}_3$ as inert—while another emulator ranks $\mathbf{x}_3$ highest, de-emphasizes $\mathbf{x}_1$, and again sees $\mathbf{x}_2$ as moderately important. Such a discrepancy suggests a lack of identifiability between $\mathbf{x}_1$ and $\mathbf{x}_3$, and highlights how choosing a single emulator might obscure meaningful structure in the data. Had we selected only one model, we might have confidently reported that $\mathbf{x}_1$ does not matter—when in fact, the full Rashomon set tells a more nuanced story.

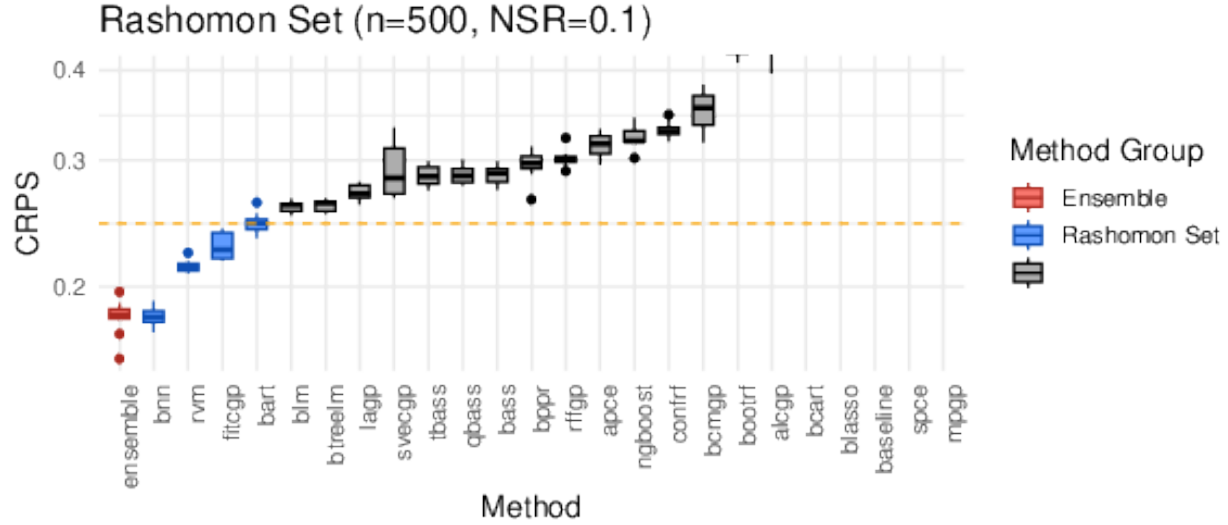


Figure 1: Rashomon plot

SM 5 Additional Figures and Analysis

In this section, we illustrate how the `duqling` framework can be used to explore additional metrics and visualizations beyond those presented in the main text. See Section SM1 and SM2 for details on installing/loading `duqling` and the simulation study objects.

We commonly *filter* the simulation study objects to study specific settings, e.g.,

```
filter_sim_study(duq_testfuncs, n_train=500, NSR=0)
```

The plotting and analysis functions provided by the package include

- `summarize_sim_study()`: Computes summary statistics across simulation scenarios (e.g., mean or median performance), producing a compact view of emulator performance.
- `heatmap_sim_study()`: Visualizes performance metrics (e.g., CRPS, FVU) across scenarios using heatmaps, helping identify patterns across settings.
- `rankplot_sim_study()`: Displays cumulative rank distributions, showing how often each emulator achieves a given rank or better.
- `paretoplot_sim_study()`: Plots trade-offs between metrics (typically accuracy vs. compute time), highlighting Pareto-efficient methods.
- `perfprofile_sim_study()`: Generates performance profile plots, showing the proportion of scenarios for which each method is within a factor of the best.
- `boxplots_sim_study()`: Produces boxplots of performance metrics across scenarios, useful for assessing variability and the Rashomon effect.
- `rank_sim_study()`: Computes rank-based summaries for a specified metric and appends corresponding rank columns.

- `normalize_sim_study()`: Normalizes metrics (e.g., scaling relative to variance or baseline values) to improve comparability across scenarios.
- `relativize_sim_study()`: Computes relative performance metrics by comparing each method to the best-performing method within each scenario.
- `collapse_sim_study()`: Aggregates results across replications or folds, producing a reduced data frame suitable for summary analyses.

SM 5.1 Fraction of Variance Unexplained

In the main paper, we generally analyze CRPS due to our focus on uncertainty quantification. If the focus is on pure predictive performance and not on well-calibrated predictive intervals, then mean squared error (MSE) or root MSE (RMSE) is the usual choice. The fraction of variance unexplained (FVU) is simply a scaled version of MSE which improves interpretability and allows for some level of comparison between simulation scenarios. We calculate RMSE and FVU as

$$\text{RMSE} = \sqrt{\frac{1}{n_{\text{test}}} \sum_{i=1}^{n_{\text{test}}} (y_{\text{test},i} - \hat{y}_i)^2},$$

and

$$\text{FVU} = \frac{\frac{1}{n_{\text{test}}} \sum_{i=1}^{n_{\text{test}}} (y_{\text{test},i} - \hat{y}_i)^2}{\frac{1}{n_{\text{test}}-1} \sum_{i=1}^{n_{\text{test}}} (y_{\text{test},i} - \bar{y}_{\text{test}})^2} = \frac{\text{RMSE}^2}{s_y^2},$$

where s_y^2 is the sample variance of the test responses and $\hat{y}_1, \dots, \hat{y}_{n_{\text{test}}}$ are the emulator predictions.

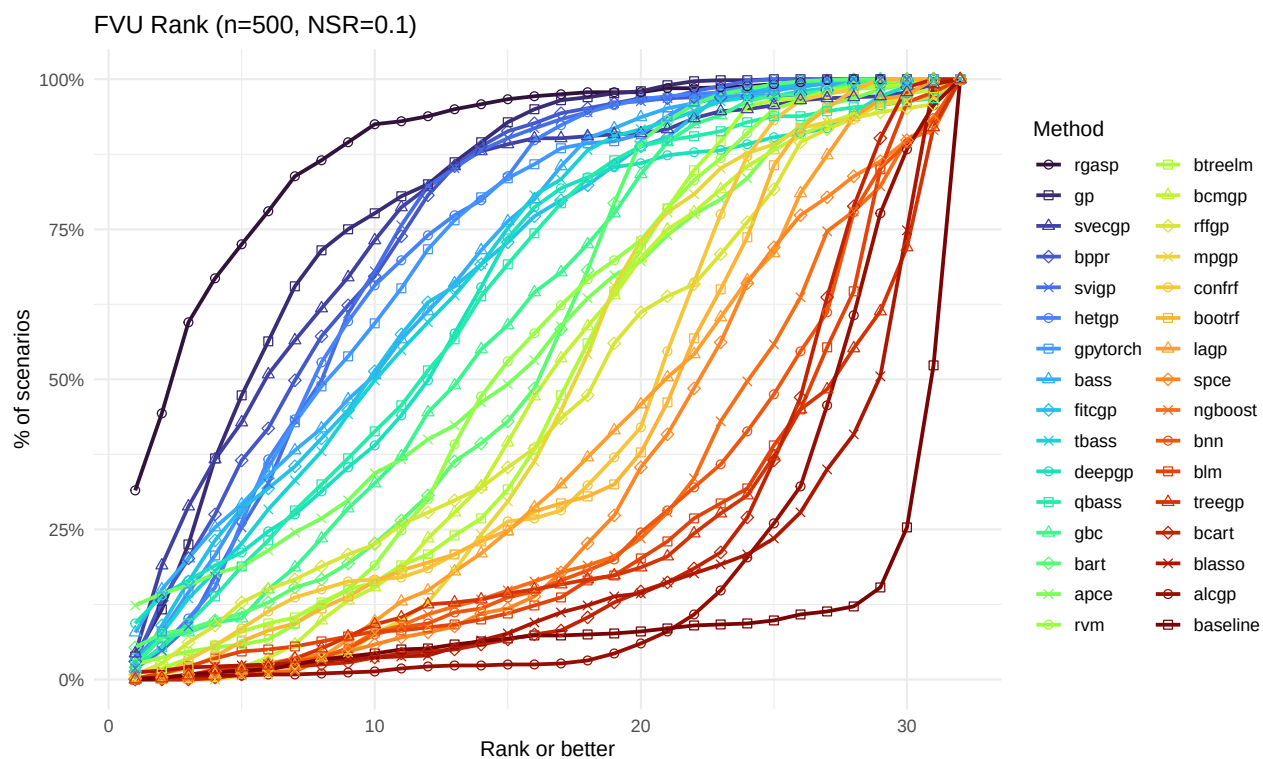
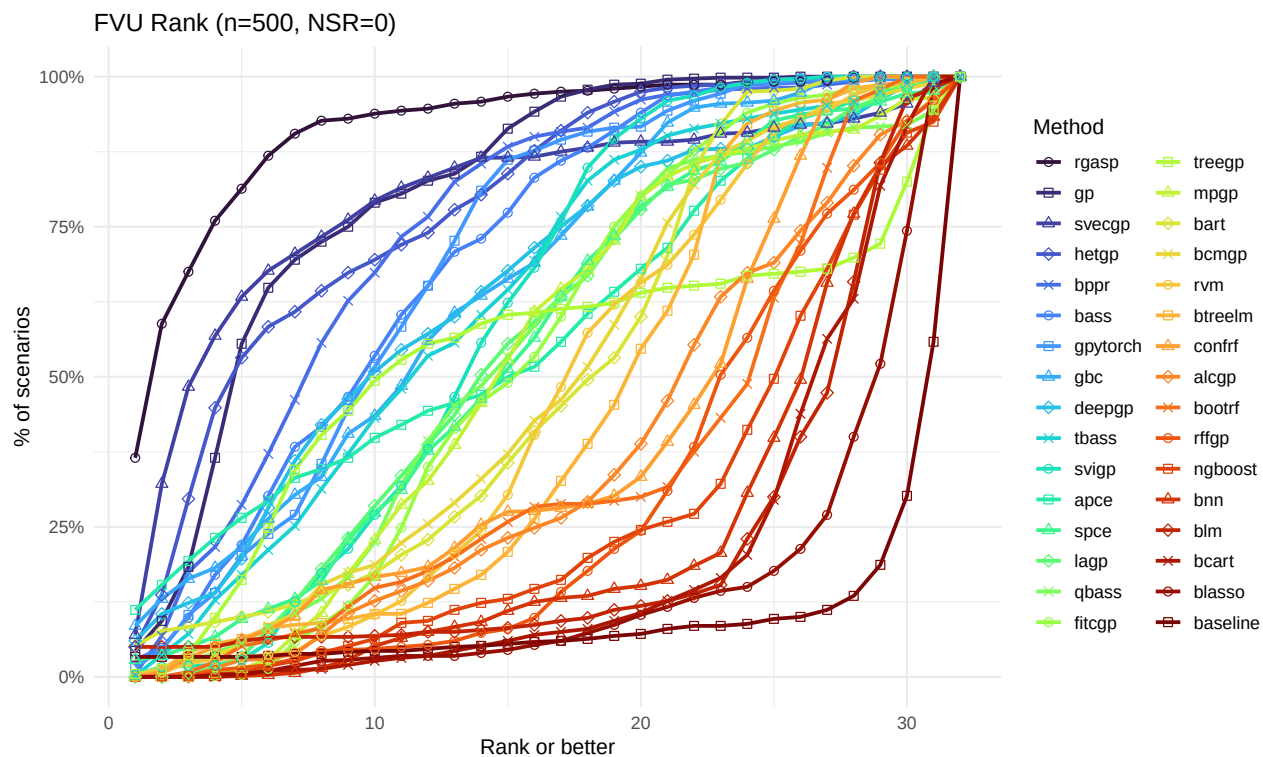
where $\hat{y}_1, \dots, \hat{y}_{n_{\text{test}}}$ are the emulator predictions. This metric (along with RMSE) is automatically calculated during `run_sim_study()` and `run_sim_study_data()`.

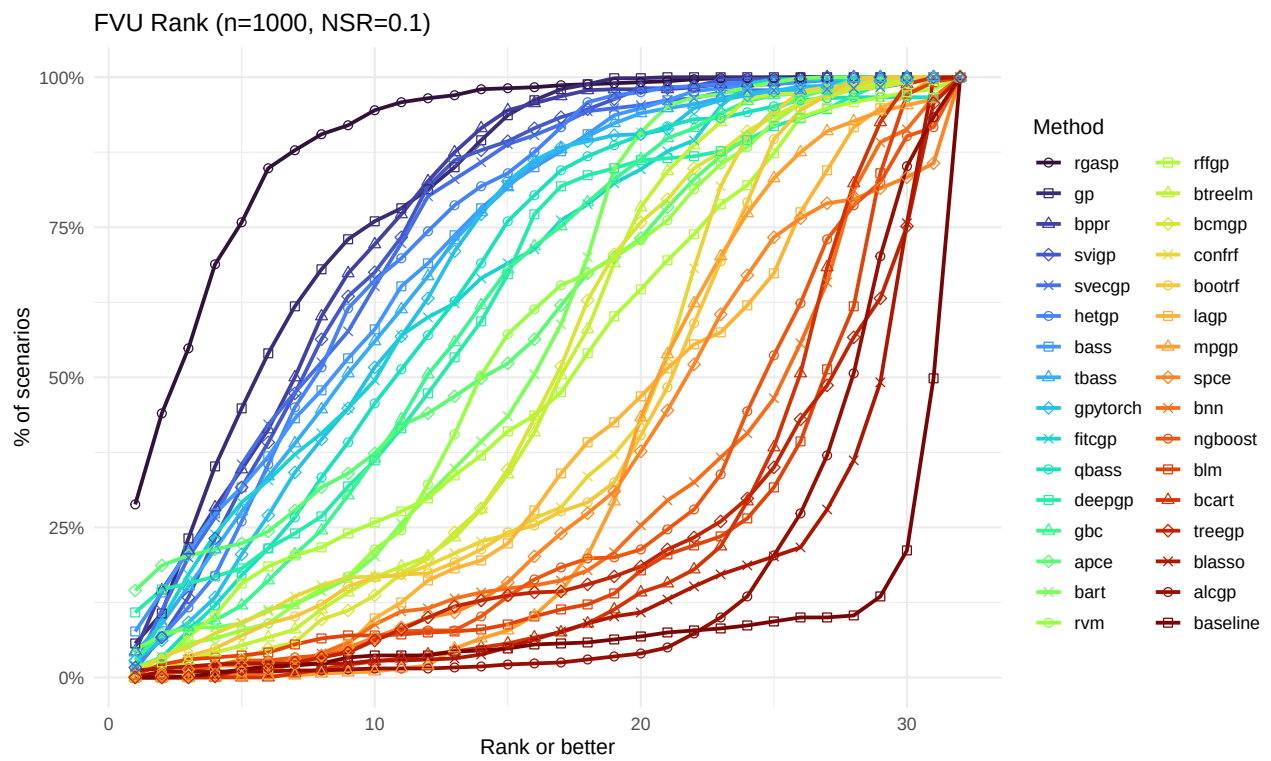
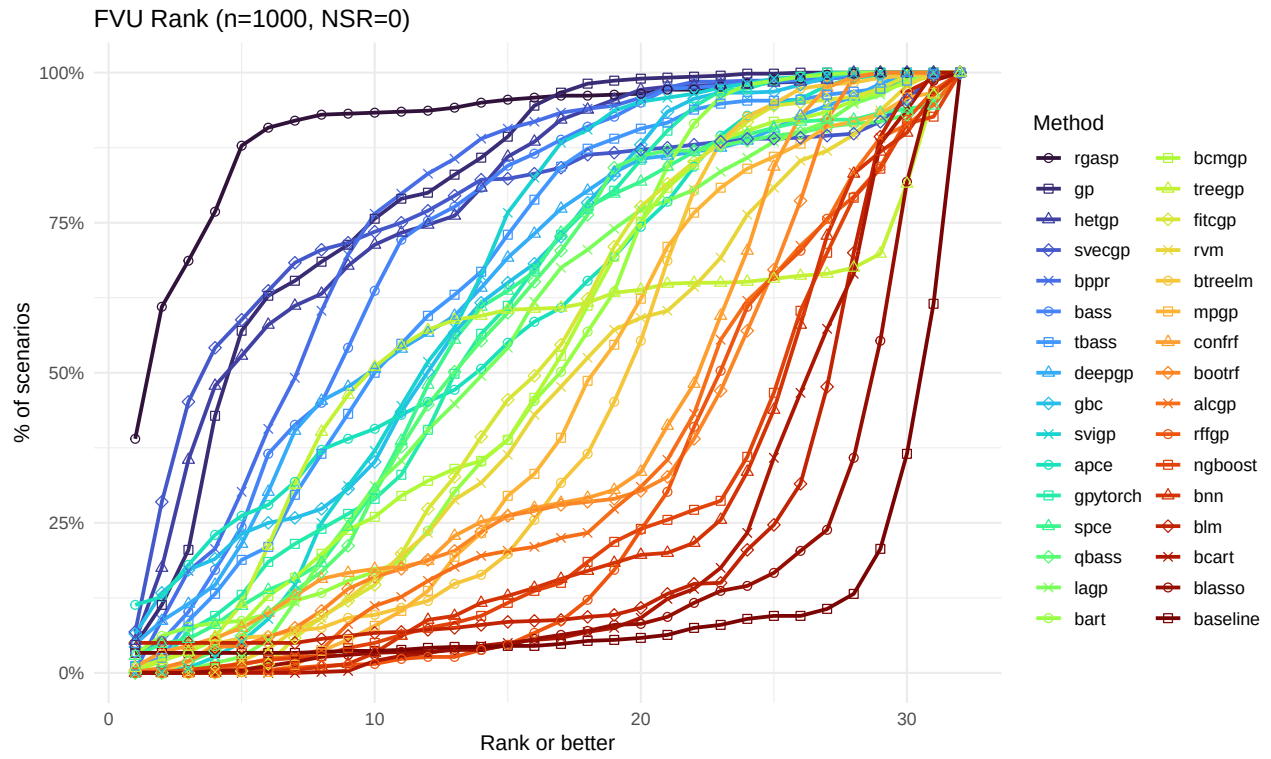
SM 5.1.1 FVU Rankplots

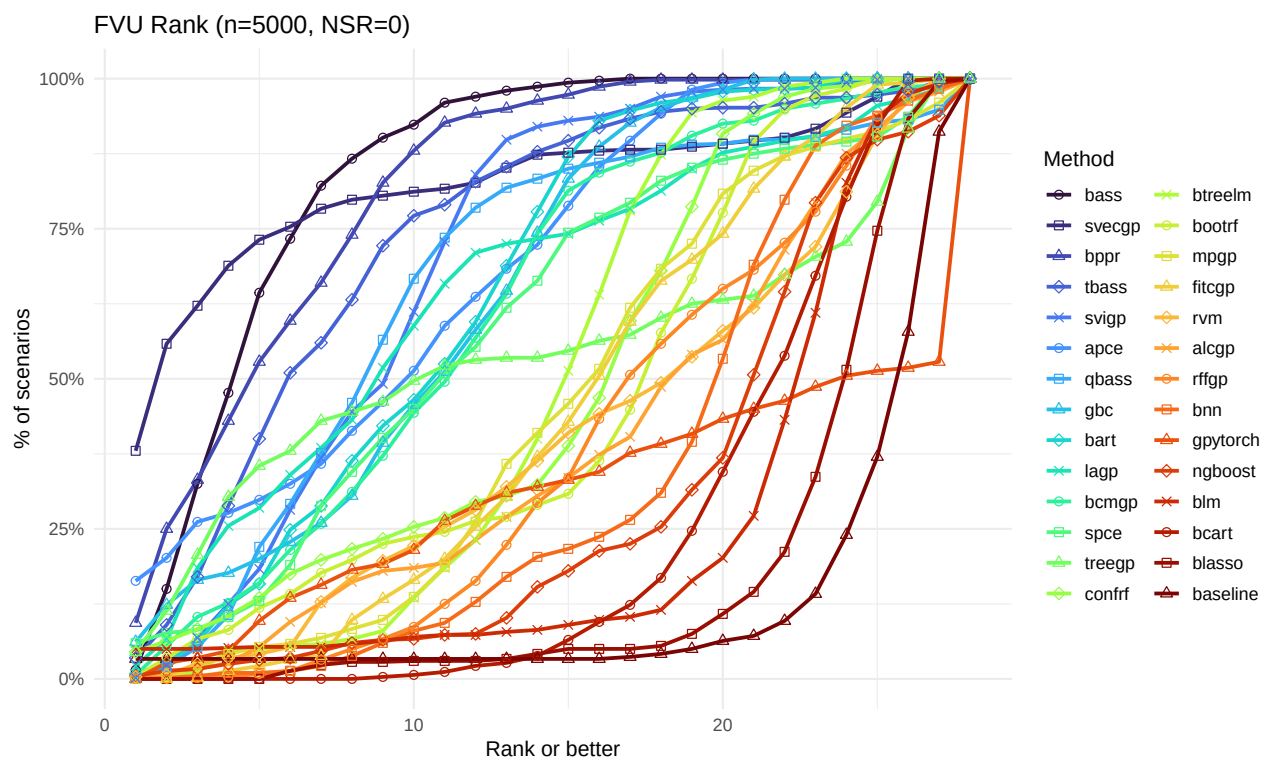
We use the following loop structure to iterate over the five synthetic simulation settings.

```
n_train_vec <- c(500, 500, 1000, 1000, 5000)
NSR_vec      <- c(0, 0.1, 0, 0.1, 0)
for(i in 1:5){
  title <- paste0("FVU Rank (n=", n_train_vec[i], ", NSR=", NSR_vec[i], ")")
  tmp <- filter_sim_study(duq_testfuncs,
                        n_train = n_train_vec[i], NSR = NSR_vec[i])
  p <- rankplot_sim_study(tmp, metric="FVU", title=title)

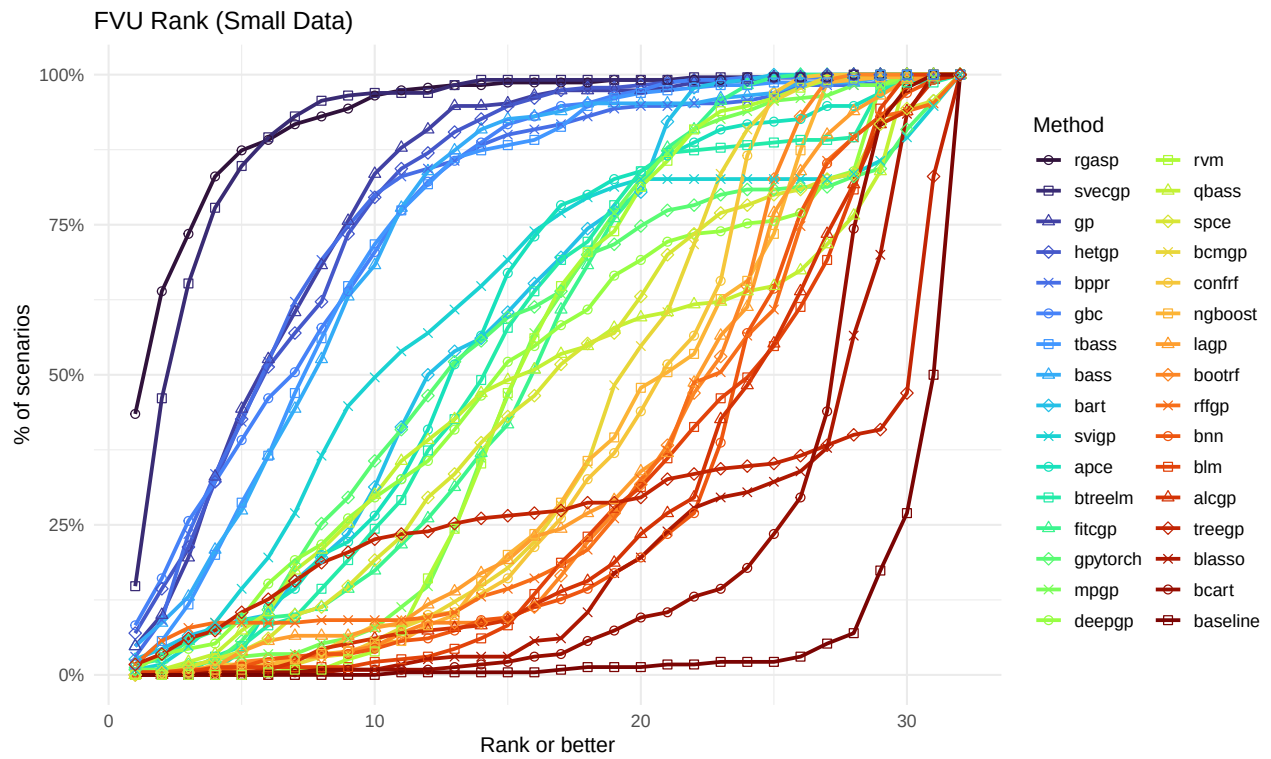
  # Adjust sizing
  p <- p + ggplot2::theme_minimal(base_size=12)
  plot(p)
}
```

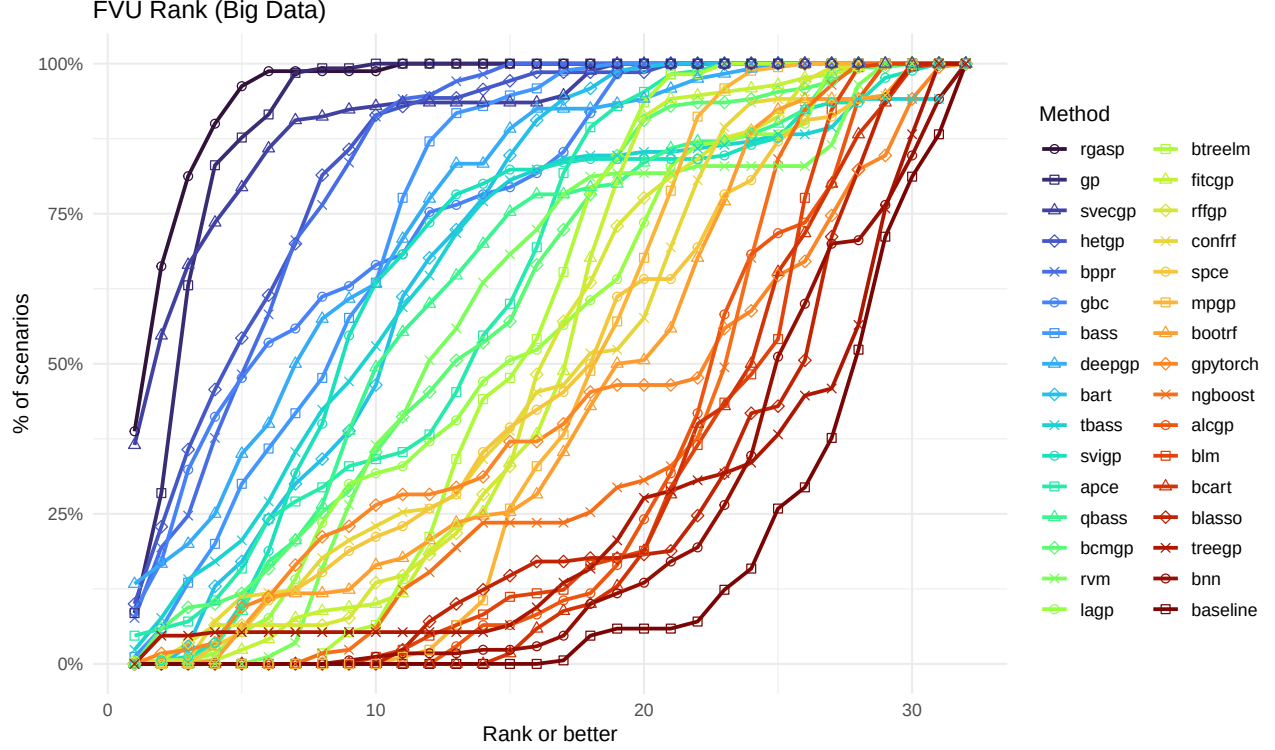




```
# Small data
title <- paste0("FVU Rank (Small Data)")
tmp <- filter_sim_study(duq_realdata,
                        expr = n_train <= 1500)
p <- rankplot_sim_study(tmp, metric="FVU", title=title)
p <- p + ggplot2::theme_minimal(base_size=12)
plot(p)
```



```
# Large data
title <- paste0("FVU Rank (Big Data)")
tmp <- filter_sim_study(duq_realdata,
                        expr = n_train > 1500)
p <- rankplot_sim_study(tmp, metric="FVU", title=title)
p <- p + ggplot2::theme_minimal(base_size=12)
plot(p)
```



SM 5.1.2 FVU Performance Profiles

Performance profiles come from the literature on benchmarking for optimization research ([30]; [31]). For emulator i and simulation scenario j we define

$$t_{i,j} = \frac{\text{metric}_{i,j} + \epsilon}{\min_i \{\text{metric}_{i,j}\} + \epsilon},$$

and we define

$$\rho_i(\tau) = \frac{1}{J} \sum_{j=1}^J 1(t_{i,j} \leq \tau).$$

When $\epsilon = 0$, the idea is that $\rho_i(\tau)$ tells us the proportion of the time that emulator i is within a factor τ of the best performing emulator across the J different simulation scenarios. We use a small value of $\epsilon = 0.001$ to perform a type of “Laplace smoothing”, which effectively induces the belief that values of the metric (FVU, here) less than this value are all effectively equally good.

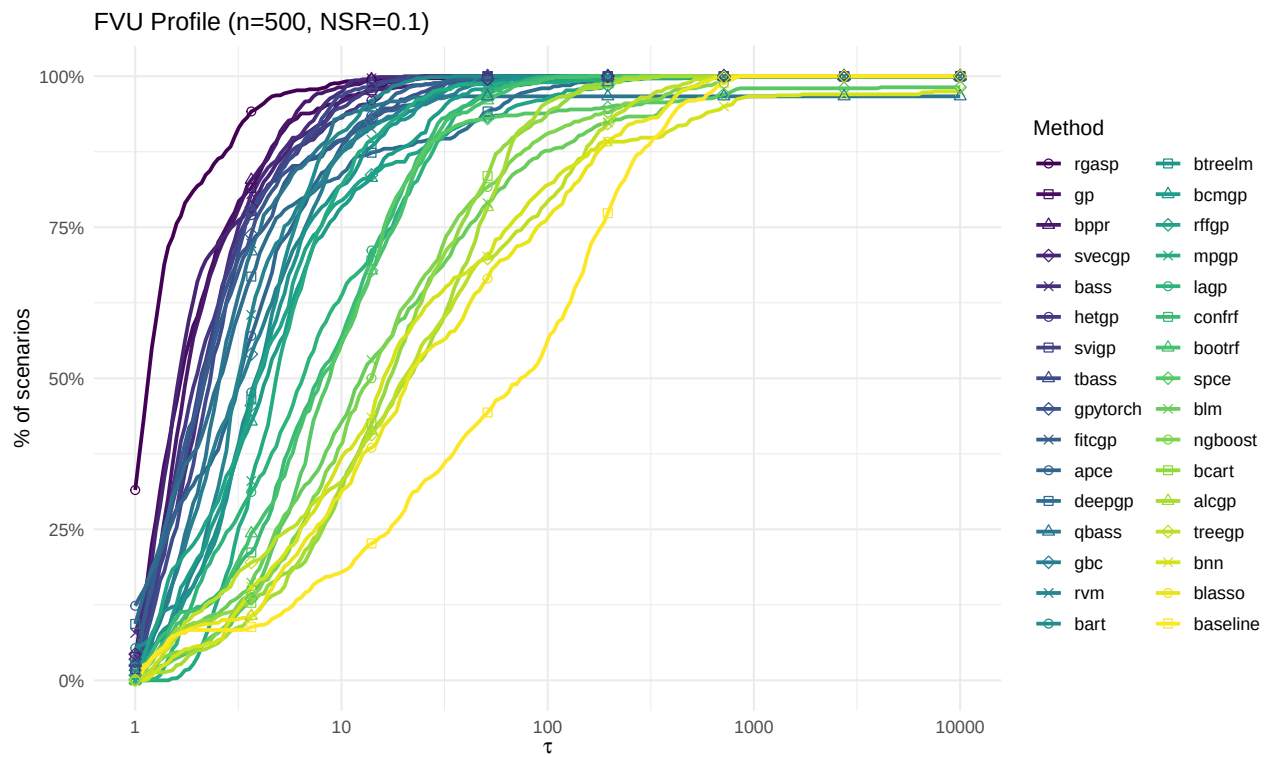
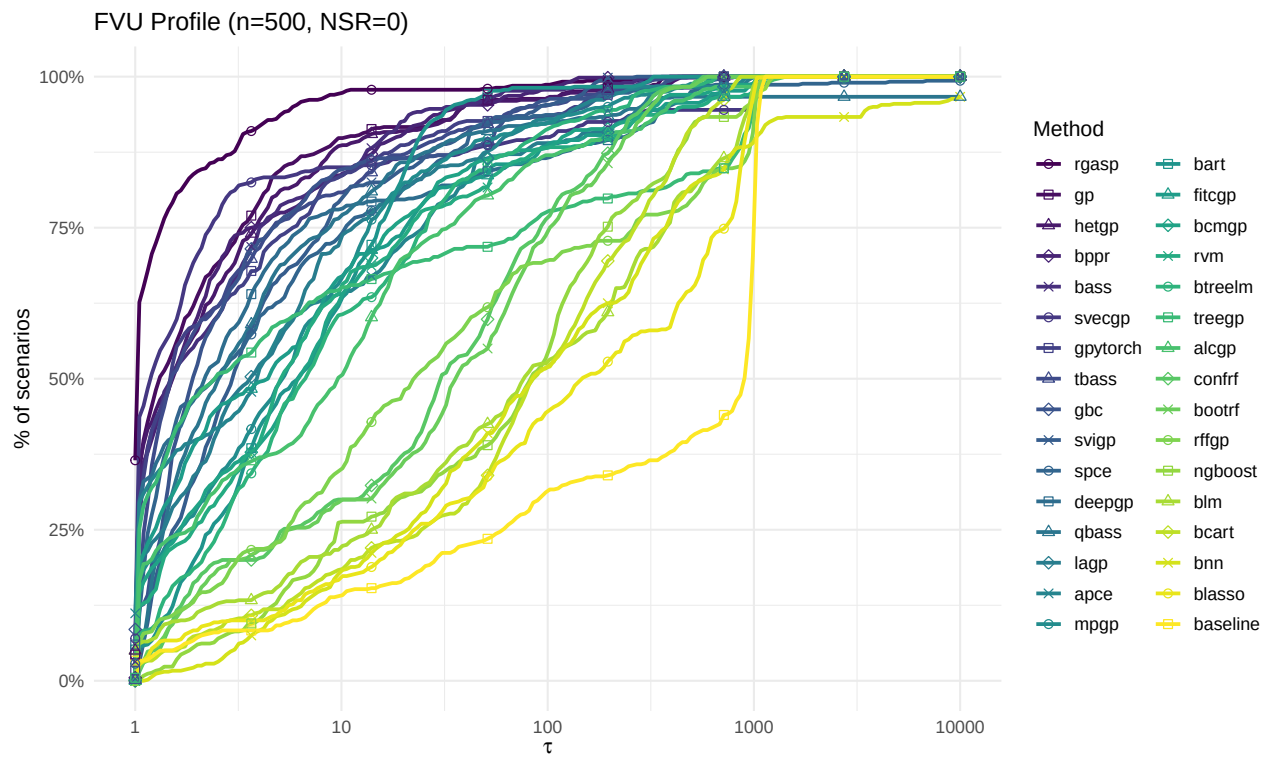
```
n_train_vec <- c(500, 500, 1000, 1000, 5000)
NSR_vec      <- c(0, 0.1, 0, 0.1, 0)
for(i in 1:5){
  title <- paste0("FVU Profile (n=", n_train_vec[i], ", NSR=", NSR_vec[i], ")")
  tmp <- filter_sim_study(duq_testfuncs,
    n_train = n_train_vec[i], NSR = NSR_vec[i])
  p <- perfprofile_sim_study(tmp, metric="FVU",
    relative_epsilon = 0.001, tau_lim=c(1, 10000),
    title=title, color_scale = "viridis")

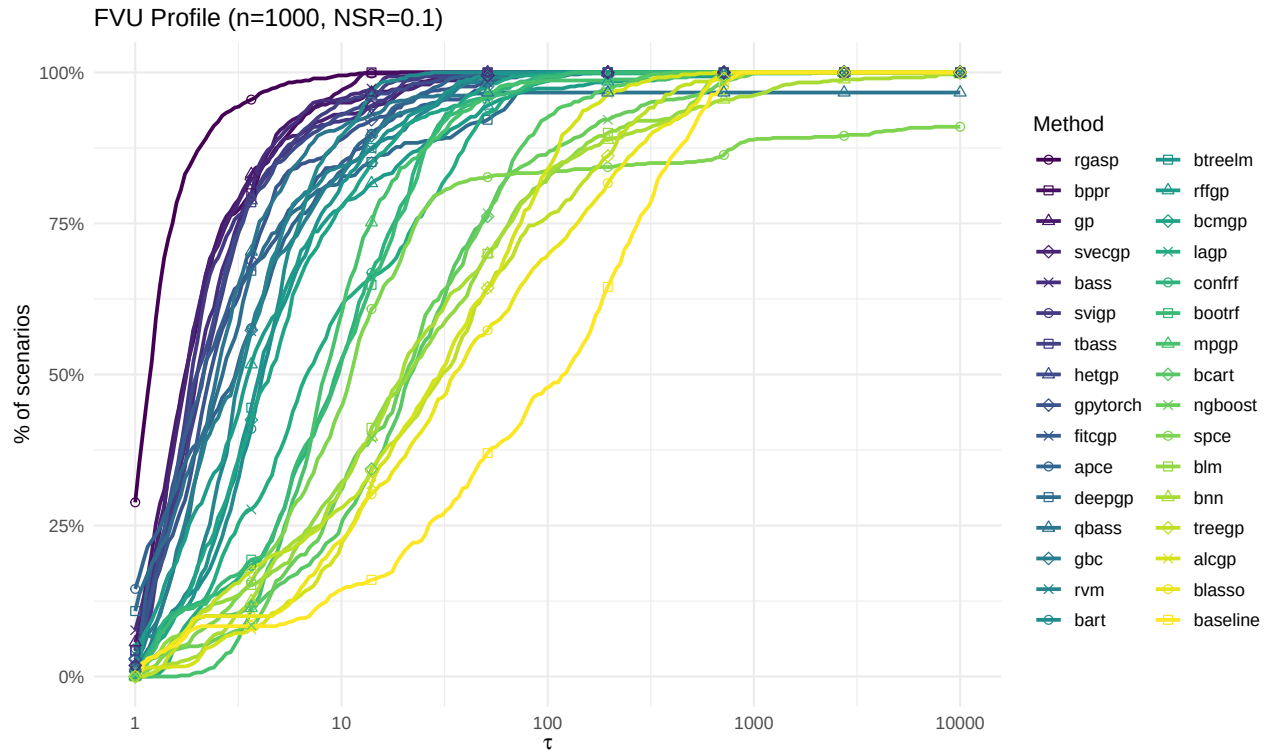
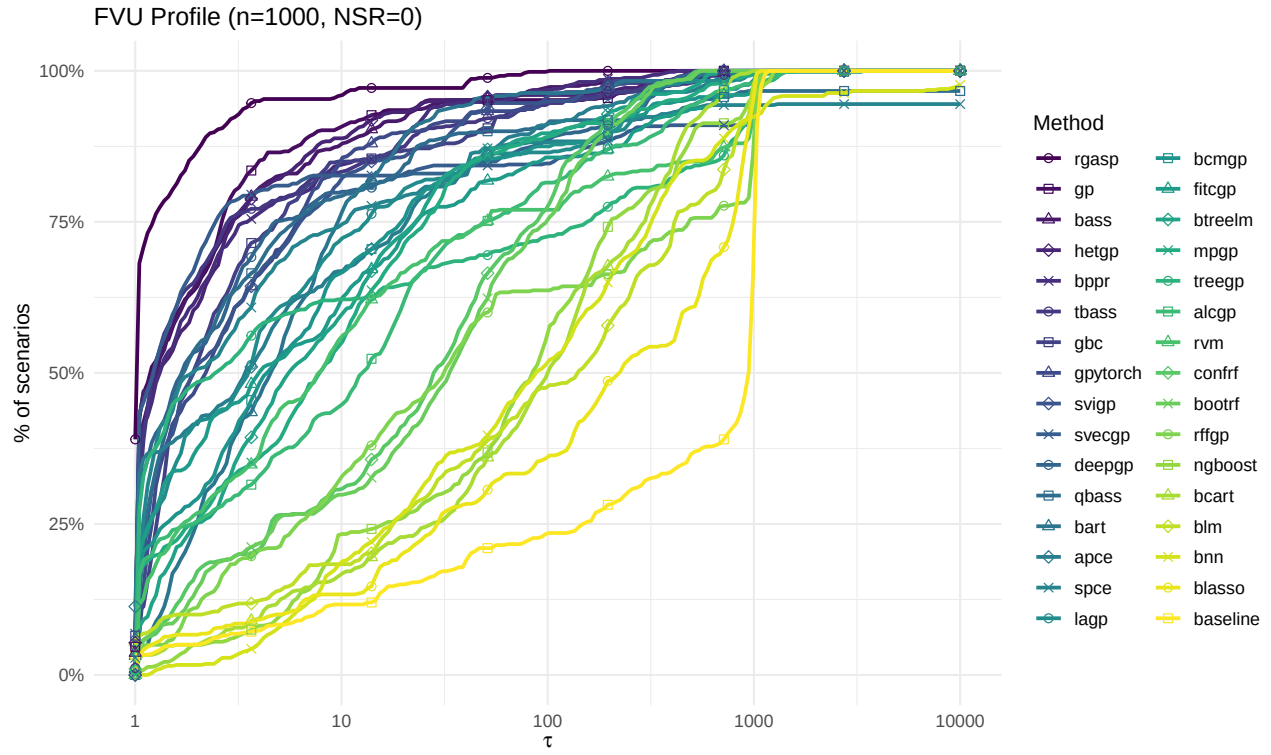
  # Adjust sizing
```

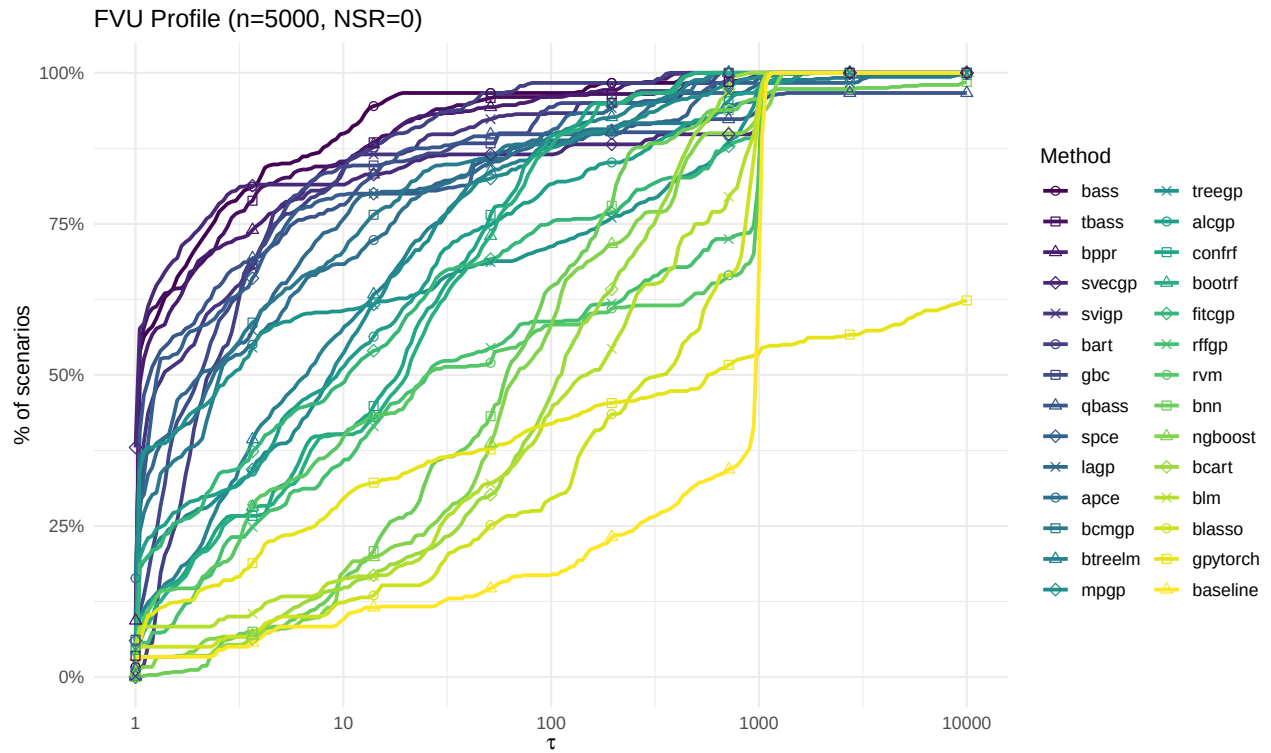
```

p <- p + ggplot2::theme_minimal(base_size=12)
plot(p)
}

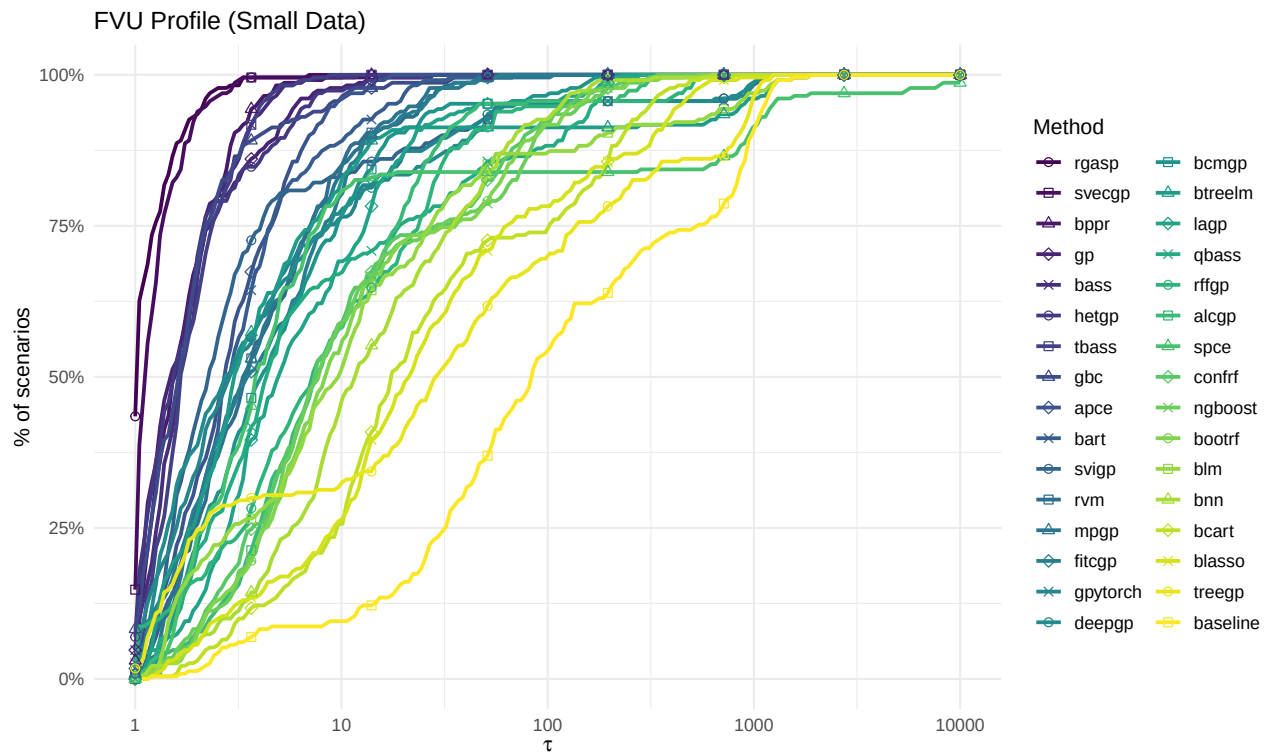
```



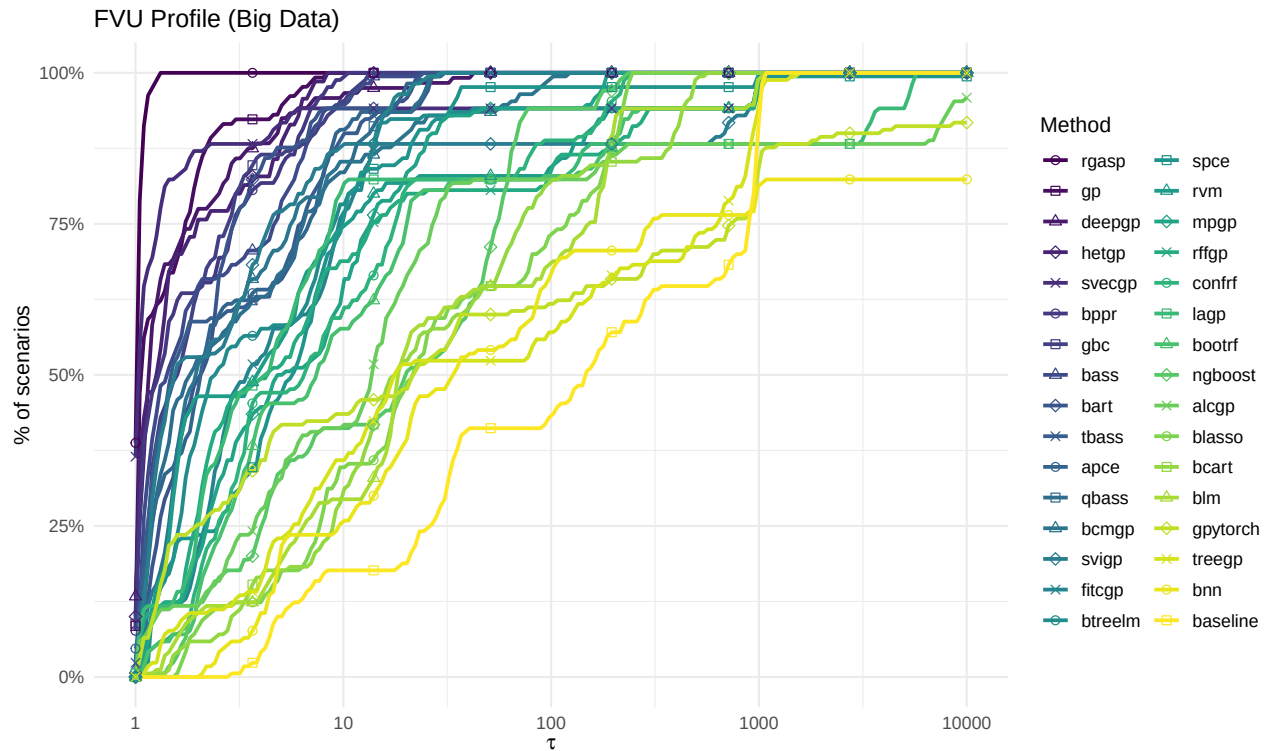




```
# Small data
title <- paste0("FVU Profile (Small Data)")
tmp <- filter_sim_study(duq_realdata,
  expr = n_train <= 1500)
p <- perfprofile_sim_study(tmp, metric="FVU",
  relative_epsilon = 0.001, tau_lim=c(1, 10000),
  title=title, color_scale = "viridis")
p <- p + ggplot2::theme_minimal(base_size=12)
plot(p)
```

```
# Large data
title <- paste0("FVU Profile (Big Data)")
tmp <- filter_sim_study(duq_realdata,
                        expr = n_train > 1500)
p <- perfprofile_sim_study(tmp, metric="FVU",
                           relative_epsilon = 0.001, tau_lim=c(1, 10000),
                           title=title, color_scale = "viridis")
p <- p + ggplot2::theme_minimal(base_size=12)
plot(p)
```



SM 5.1.3 FVU Heatmaps

We use a custom colorbar scale to make the figures more readable.

```
custom <- function(xx){
  log10(pmin(1, pmax(0.001, xx)))
}

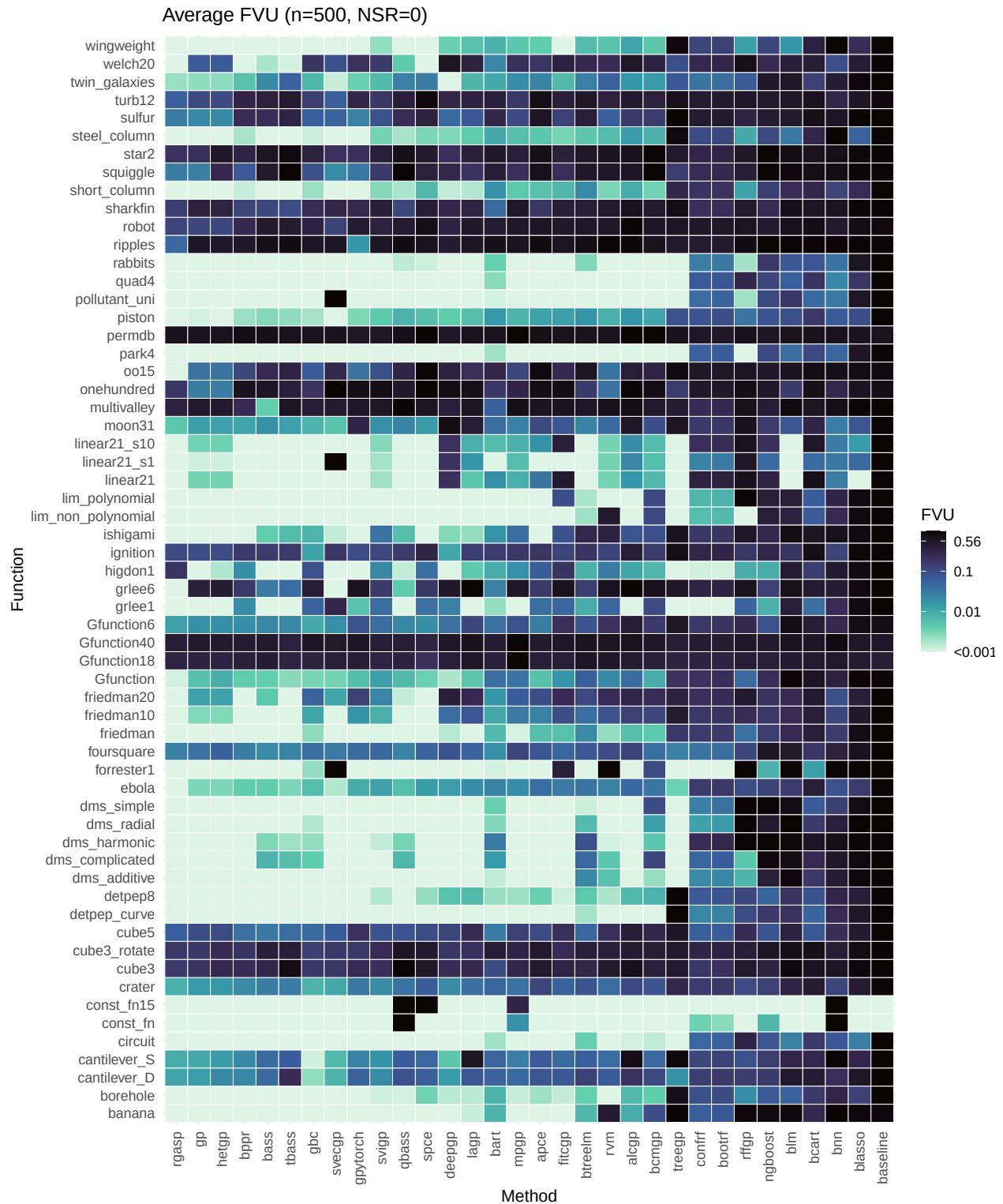
n_train_vec <- c(500, 500, 1000, 1000, 5000)
NSR_vec      <- c(0, 0.1, 0, 0.1, 0)
for(i in 1:5){
  title <- paste0("Average FVU (n=", n_train_vec[i], ", NSR=", NSR_vec[i], ")")
  tmp <- filter_sim_study(duq_testfuncs,
    n_train = n_train_vec[i], NSR = NSR_vec[i])
  p <- heatmap_sim_study(tmp, metric="FVU", y_scale_fun = custom,
    colorbar_labels =
      list(breaks=log10(c(0.001, 0.01, 0.1, 0.56, 1)),
        labels=c("<0.001", "0.01", "0.1", "0.56", "")),
    color_scale="mako",
    title=title)

  # Adjust sizing
  p <- p + ggplot2::theme_minimal(base_size=12) +
    ggplot2::theme(
      axis.text.x = ggplot2::element_text(
        angle = 90,
        vjust = 0.5,
```

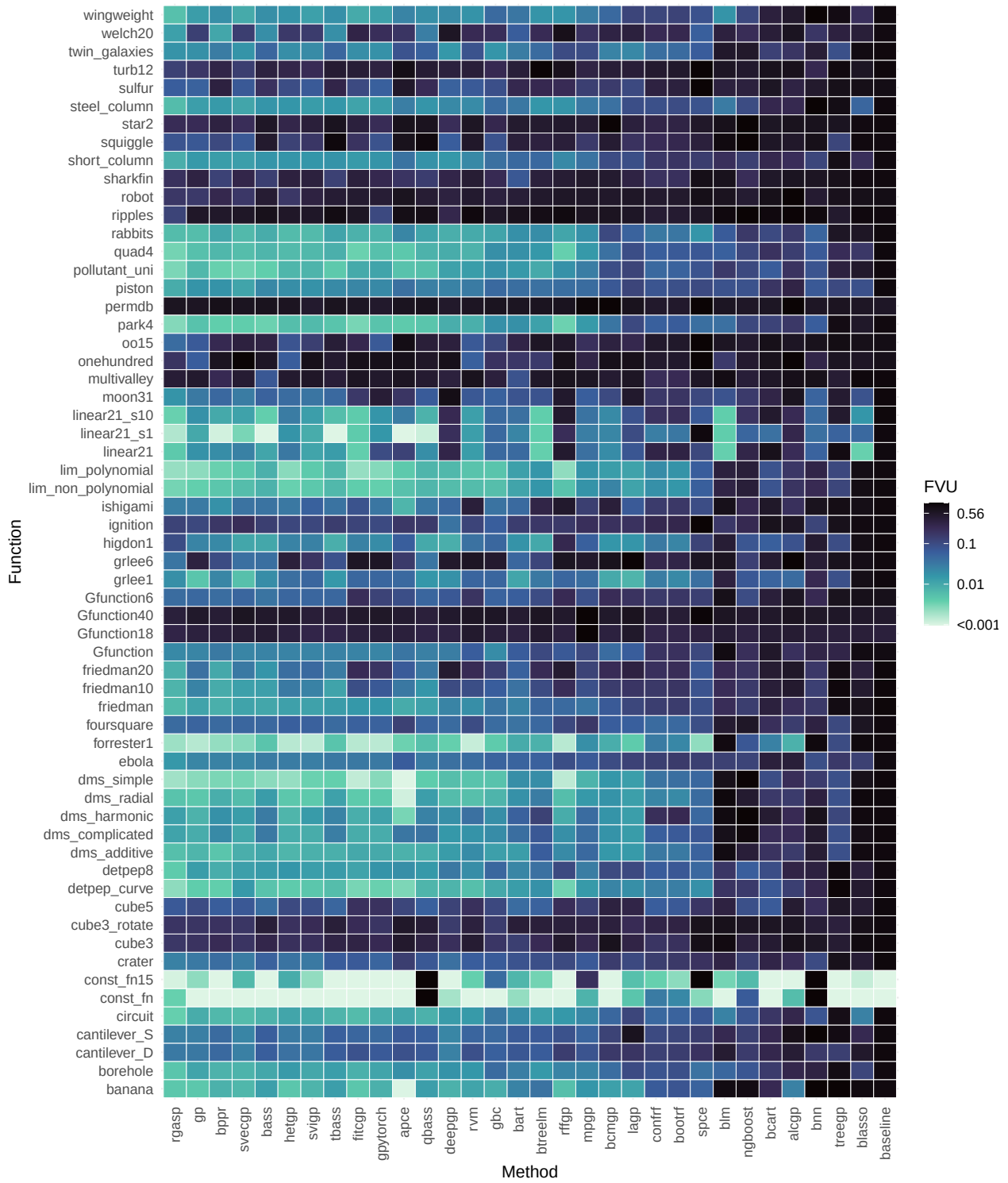
```

    hjust = 1
  ))
  plot(p)
}

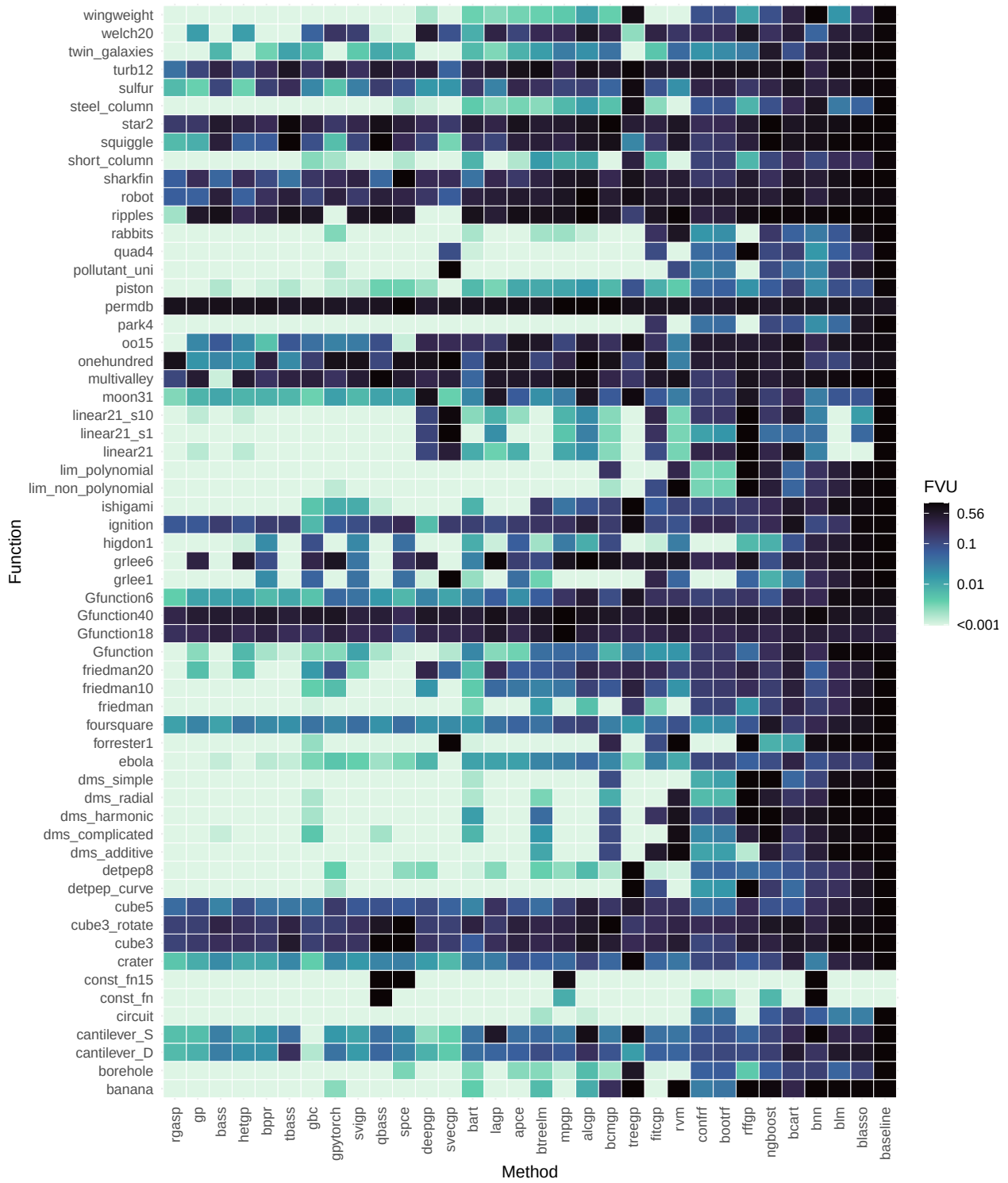
```



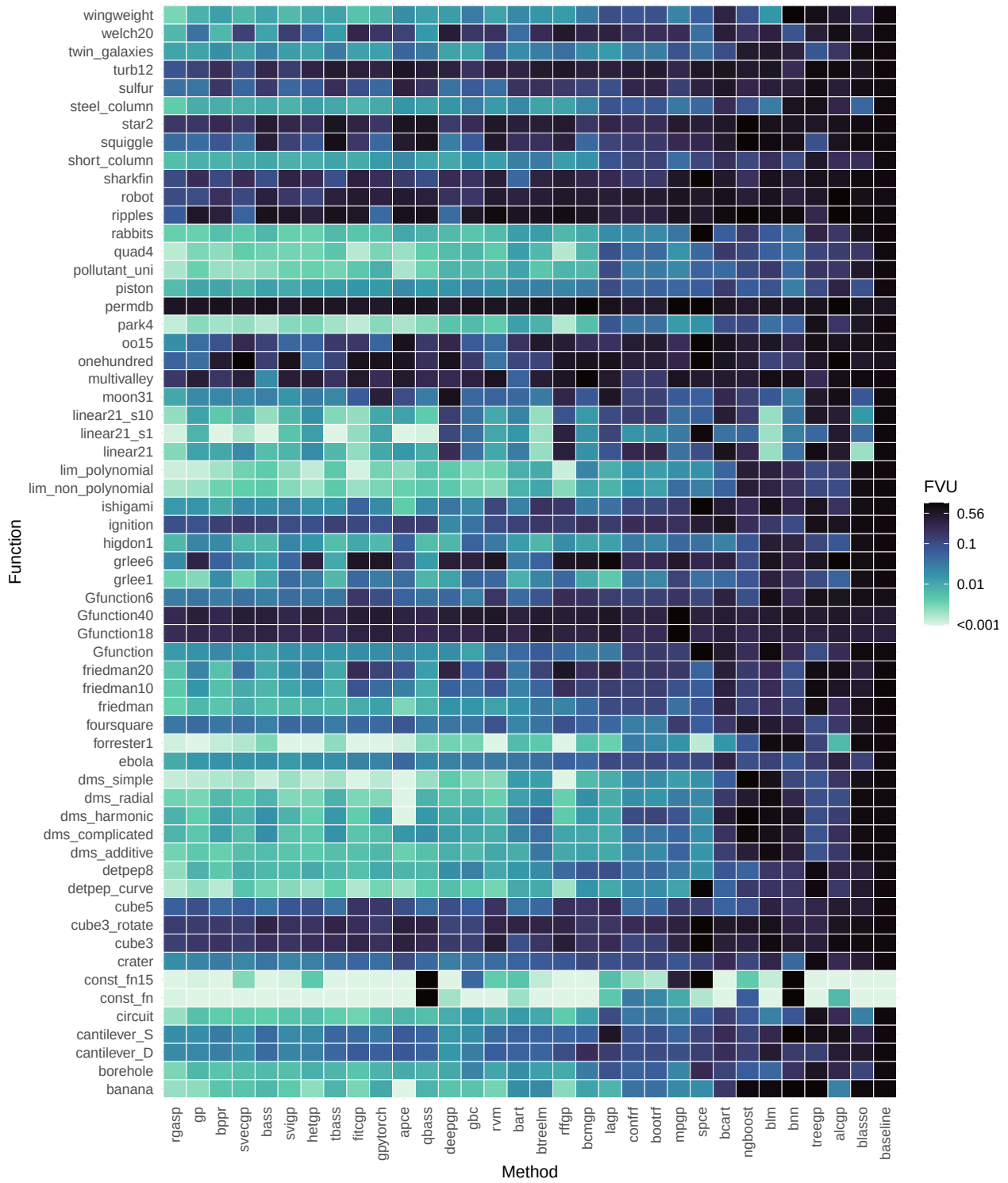
Average FVU (n=500, NSR=0.1)



Average FVU (n=1000, NSR=0)



Average FVU (n=1000, NSR=0.1)



A heatmap visualization showing the False Variable Uniqueness (FVU) values for 60 different functions (rows) evaluated by 29 different machine learning methods (columns). The color scale ranges from black (<0.001) to white (0.56), with intermediate colors representing values like 0.01 and 0.1. The functions are listed on the y-axis, and the methods are listed on the x-axis. The legend on the right indicates the FVU scale.

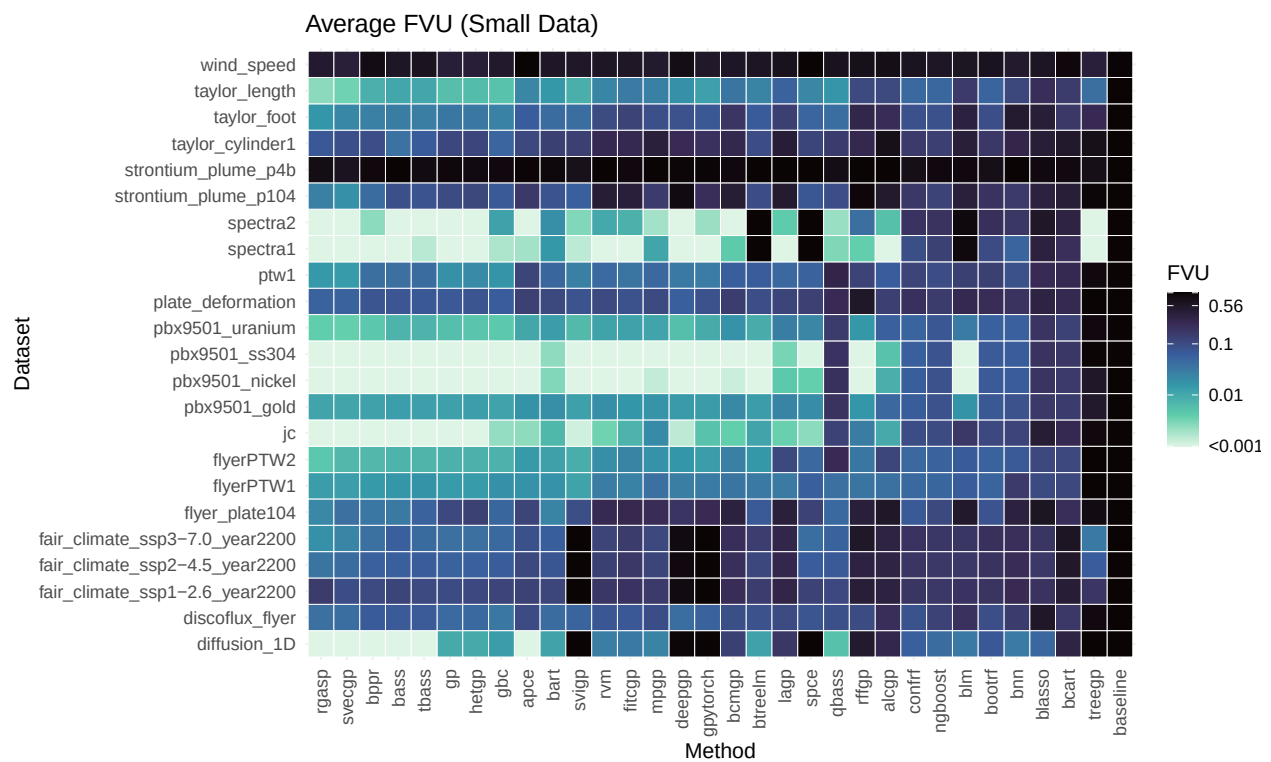
Function	bass	bpr	tbss	bart	svlgp	gbc	svecbp	qbss	spce	lagp	apce	mjpgp	btrelm	bcmgp	alcgp	cnfrf	treegp	bootrf	fitegp	rvm	rffgp	bnn	ngboost	bcart	blim	biasso	gpytorch	baseline
wingweight	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	
welch20	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	
twin_galaxies	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	
turb12	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	
sulfur	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	
steel_column	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	
star2	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	
squiggle	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	
short_column	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	
sharkfin	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	
robot	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001	
ripples	<0.001	<0.001	<																									

31

```

        colorbar_labels =
          list(breaks=log10(c(0.001, 0.01, 0.1, 0.56, 1)),
               labels=c("<0.001", "0.01", "0.1", "0.56", "")),
          color_scale="mako",
          title=title)
p <- p + ggplot2::theme_minimal(base_size=12) +
  ggplot2::theme(
    axis.text.x = ggplot2::element_text(
      angle = 90,
      vjust = 0.5,
      hjust = 1
    )
  )
plot(p)

```



```

# Large data
title <- paste0("Average FVU (Big Data)")
tmp <- filter_sim_study(duq_realddata,
  expr = n_train > 1500)
p <- heatmap_sim_study(tmp, metric="FVU", y_scale_fun = custom,
  colorbar_labels =
    list(breaks=log10(c(0.001, 0.01, 0.1, 0.56, 1)),
         labels=c("<0.001", "0.01", "0.1", "0.56", "")),
    color_scale="mako",
    title=title)
p <- p + ggplot2::theme_minimal(base_size=12) +
  ggplot2::theme(

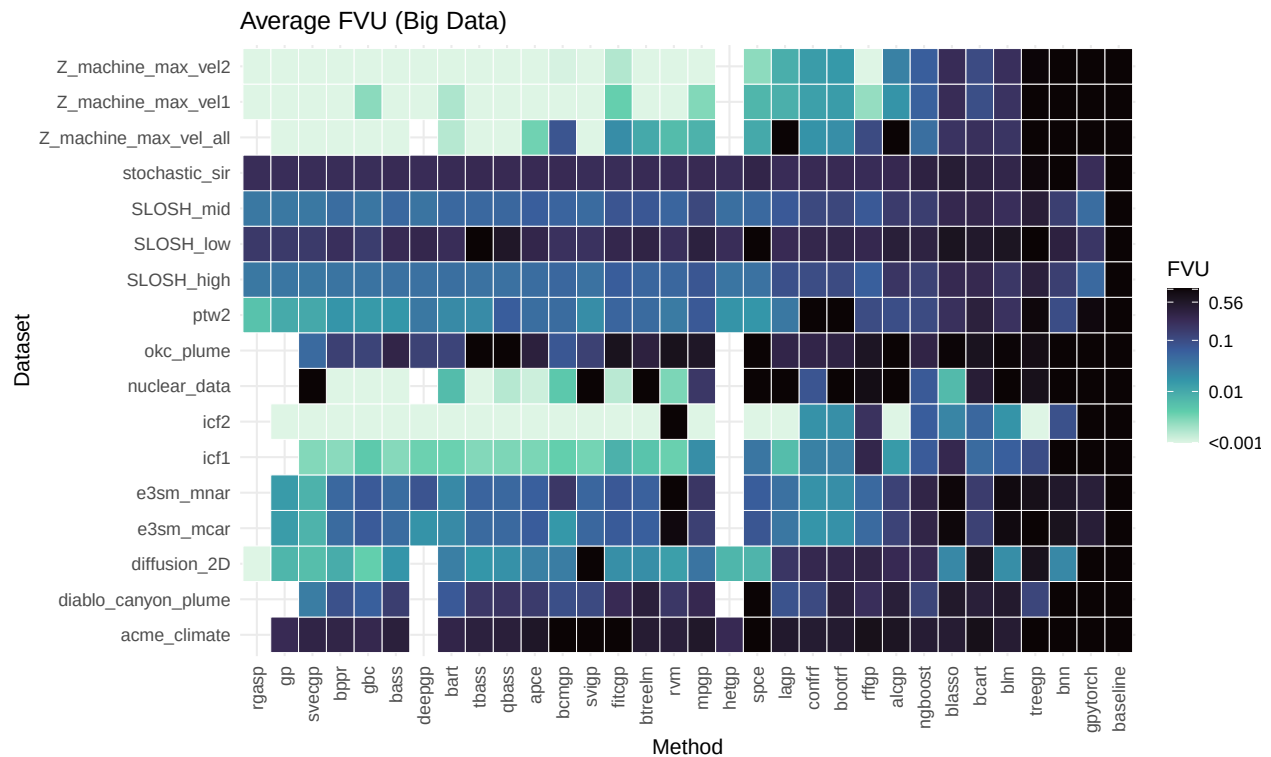
```



```

axis.text.x = ggplot2::element_text(
  angle = 90,
  vjust = 0.5,
  hjust = 1
))
plot(p)

```



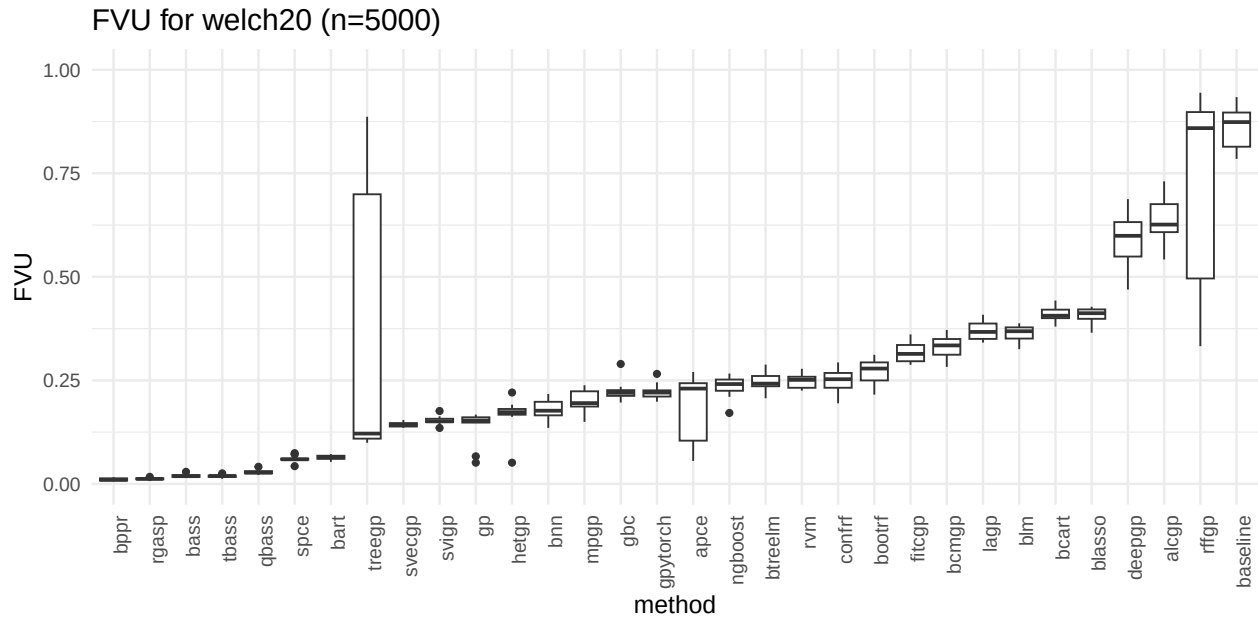
SM 5.1.4 FVU Boxplots

Boxplots are a great way to summarize a specific simulation scenario. There are thousands of possible boxplots one could make from this data, here's an example.

```

tmp <- filter_sim_study(duq_testfuncs, id = "welch20",
  n_train=500, NSR=0.1)
boxplots_sim_study(tmp, metric="FVU",
  ylim=c(0, 1), title="FVU for welch20 (n=5000)")

```

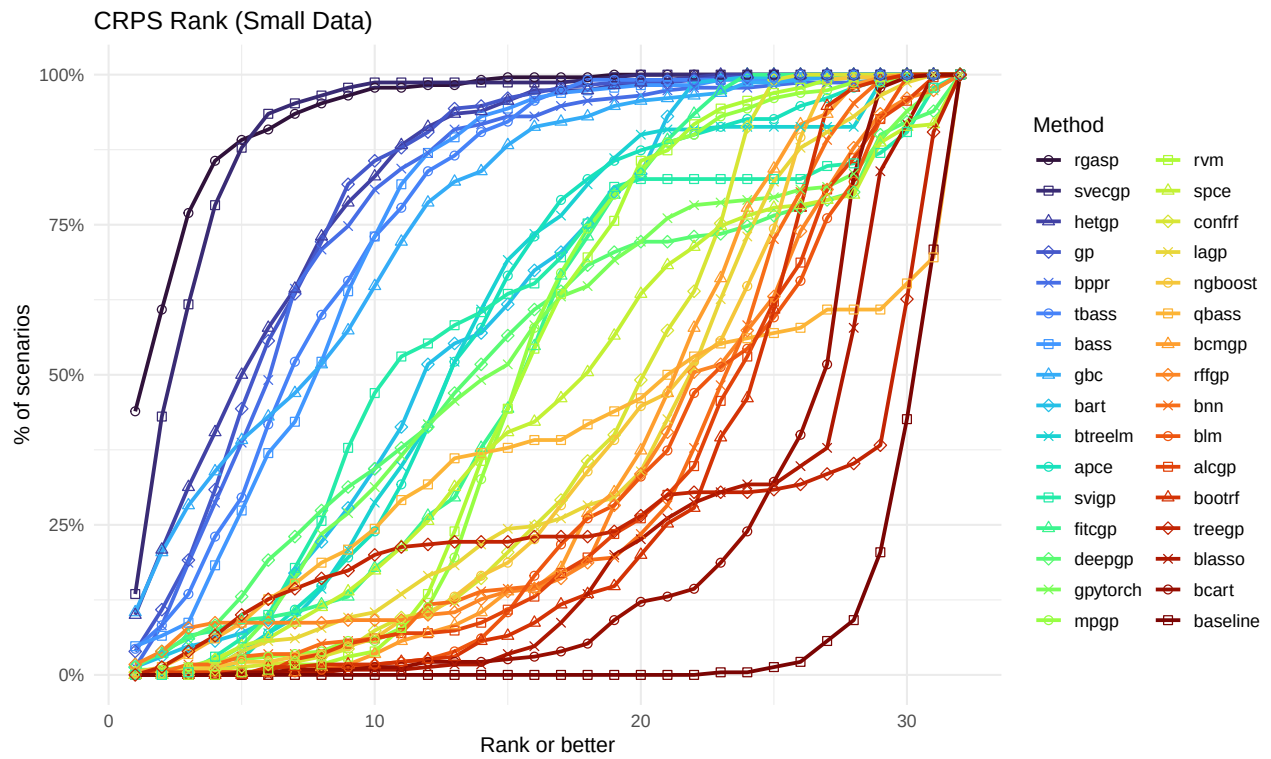


SM 5.2 Continuous Ranked Probability Scores

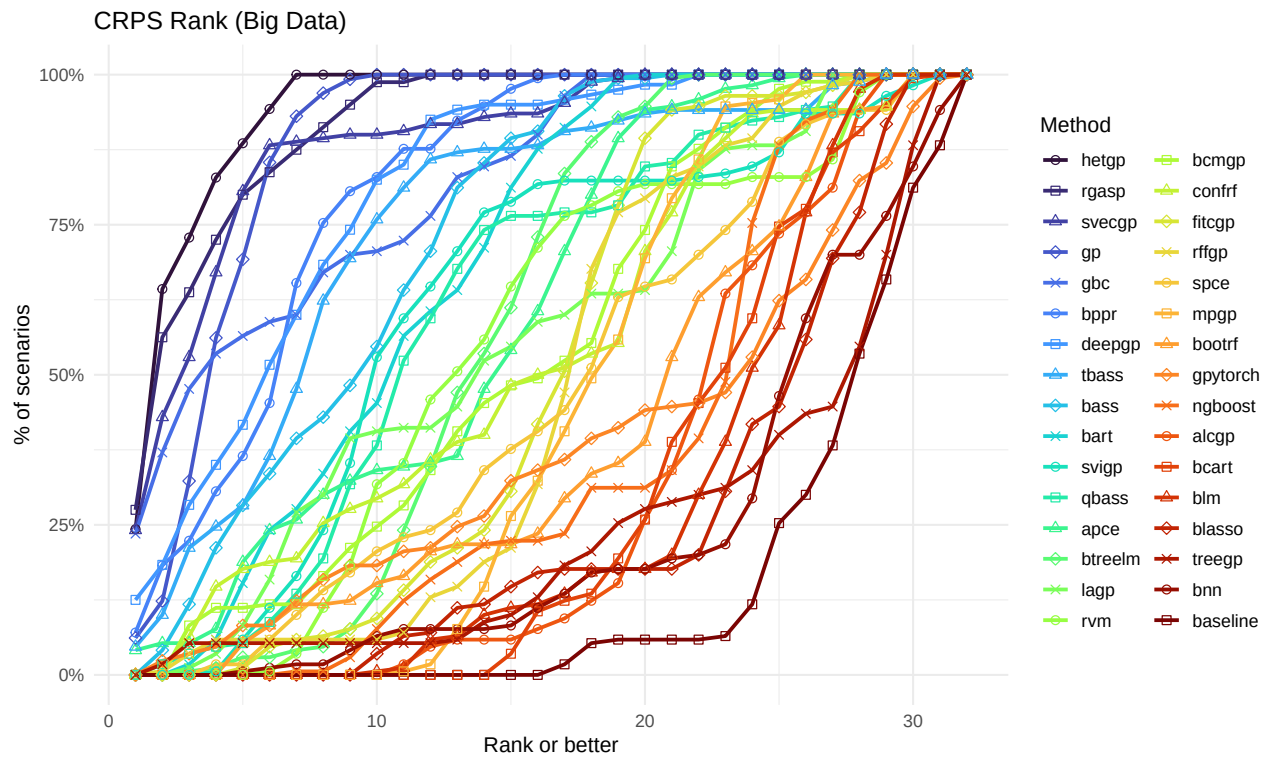
Continuous ranked probability scores (CRPS) was the primary metric used in the main manuscript, and so we won't rehash all of that here. Here we show some key figures not given in the main manuscript: (i) CRPS rankplots for the real datasets and (ii) CRPS performance profiles for the test functions.

SM 5.2.1 CRPS Rankplots (data only)

```
# Small data
title <- paste0("CRPS Rank (Small Data)")
tmp <- filter_sim_study(duq_realdata,
                        expr = n_train <= 1500)
p <- rankplot_sim_study(tmp, metric="CRPS", title=title)
p <- p + ggplot2::theme_minimal(base_size=12)
plot(p)
```



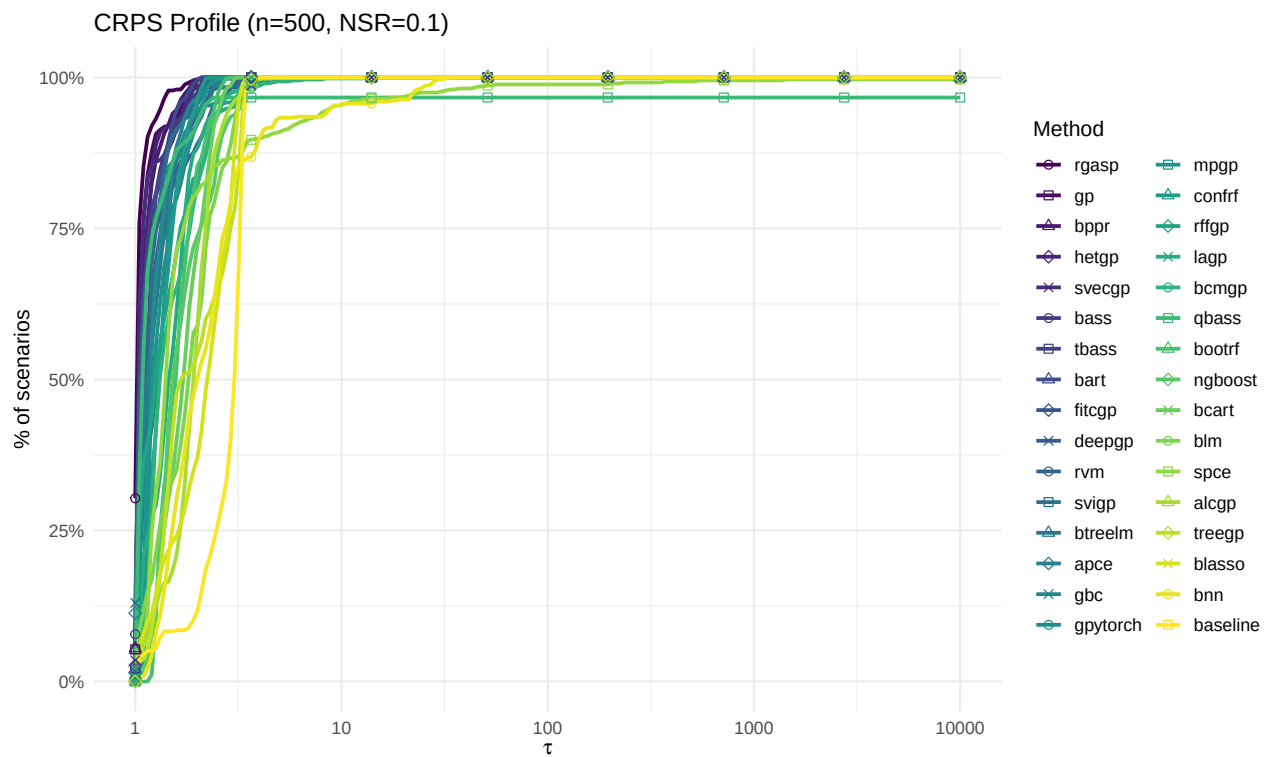
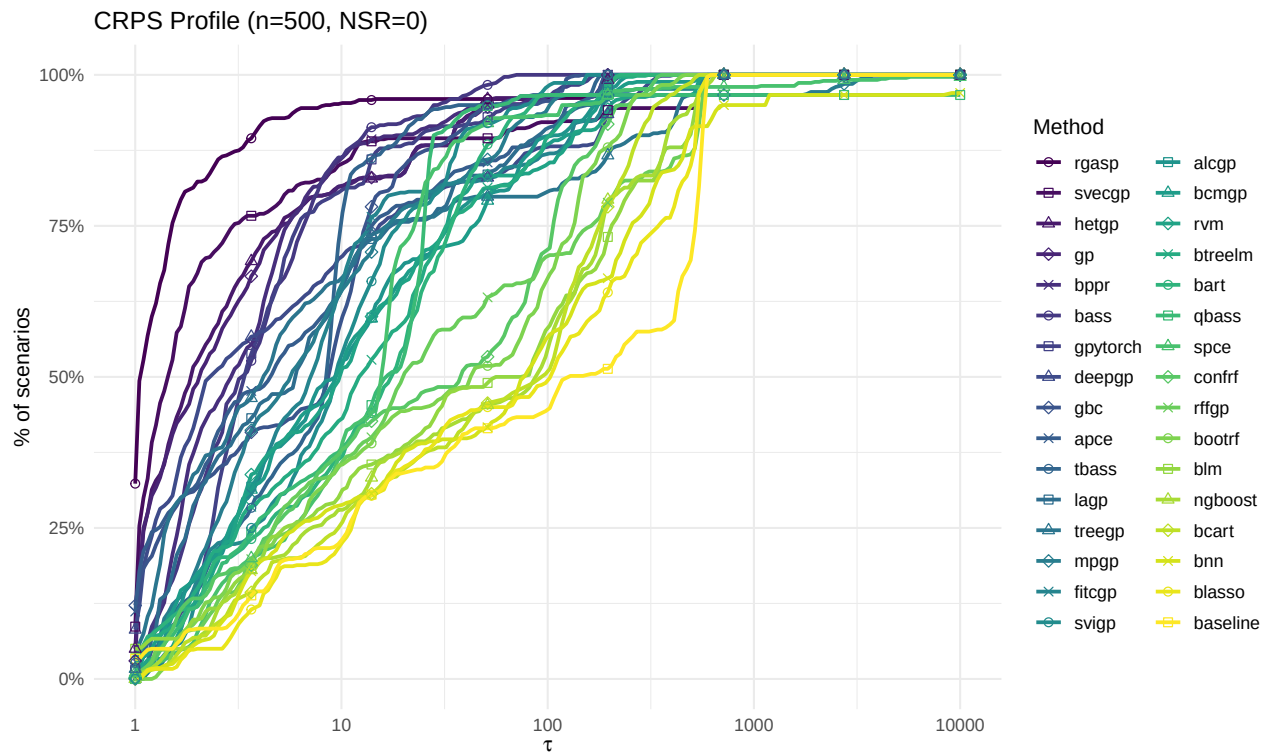
```
# Large data
title <- paste0("CRPS Rank (Big Data)")
tmp <- filter_sim_study(duq_realdata,
                        expr = n_train > 1500)
p <- rankplot_sim_study(tmp, metric="CRPS", title=title)
p <- p + ggplot2::theme_minimal(base_size=12)
plot(p)
```

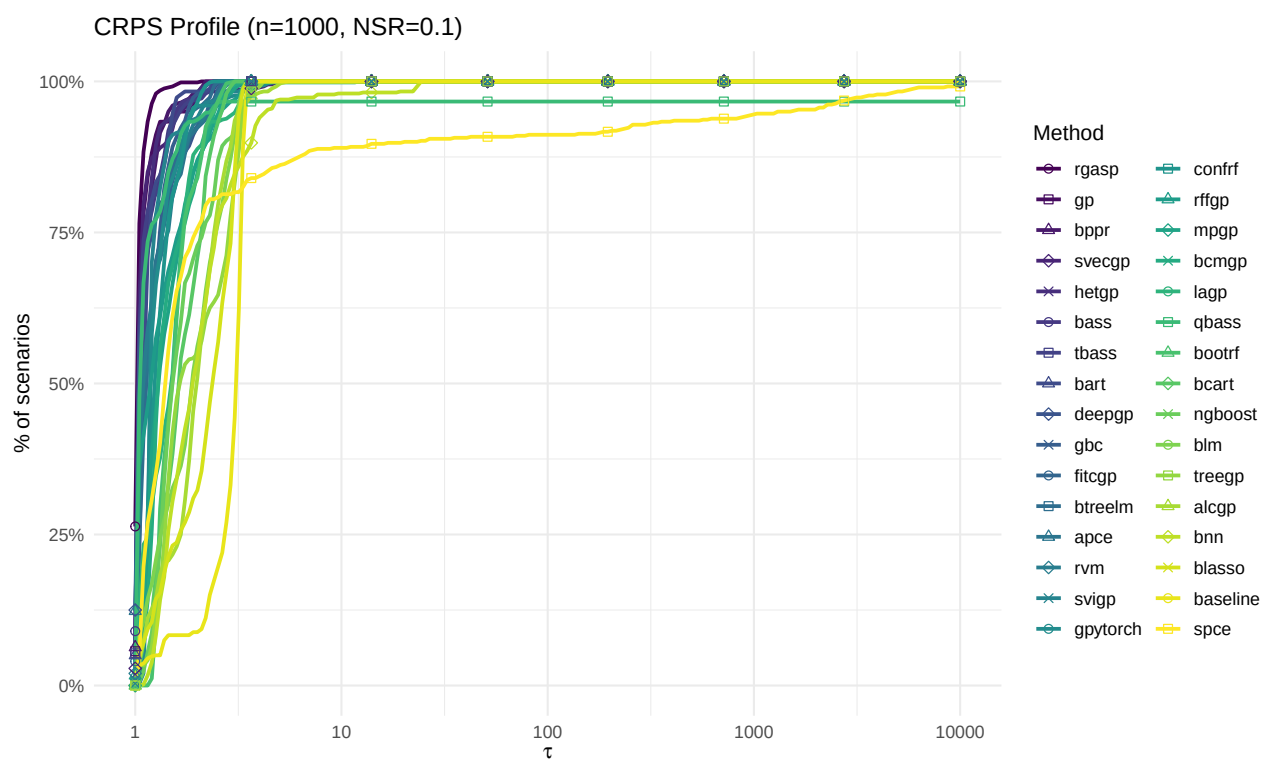
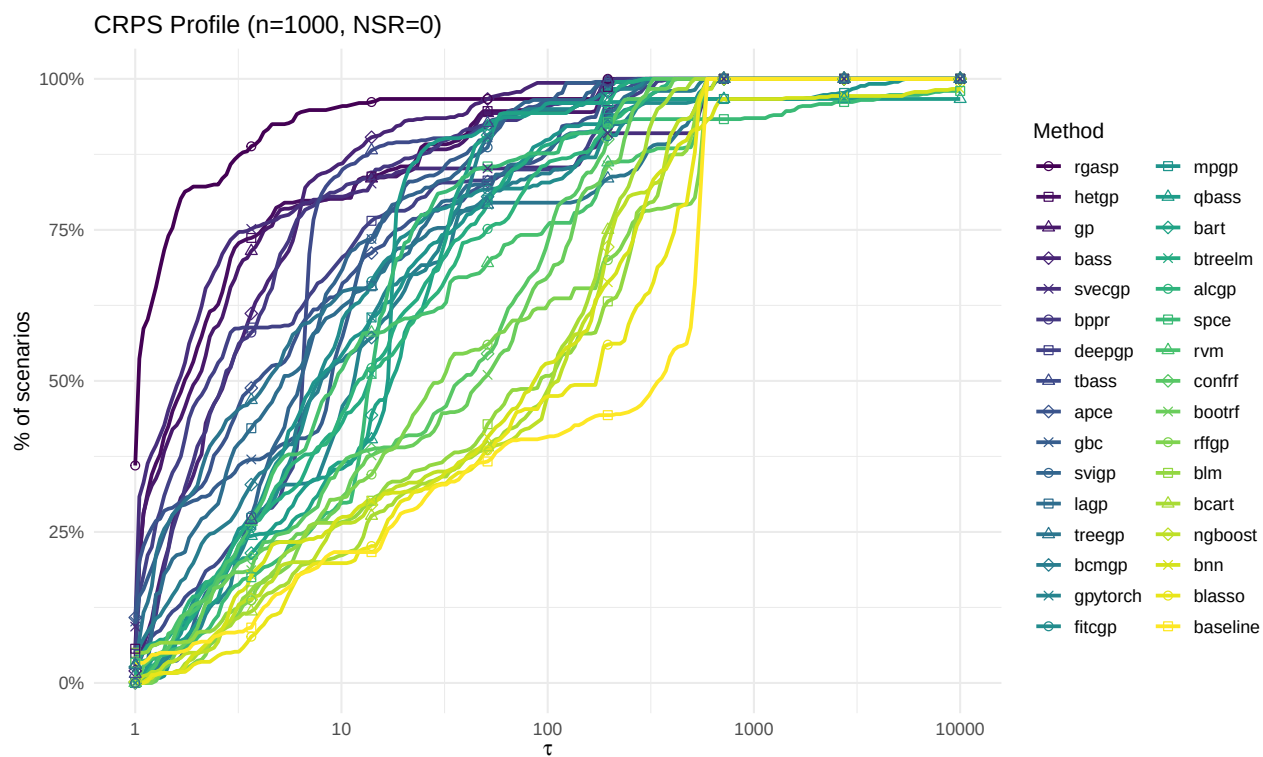


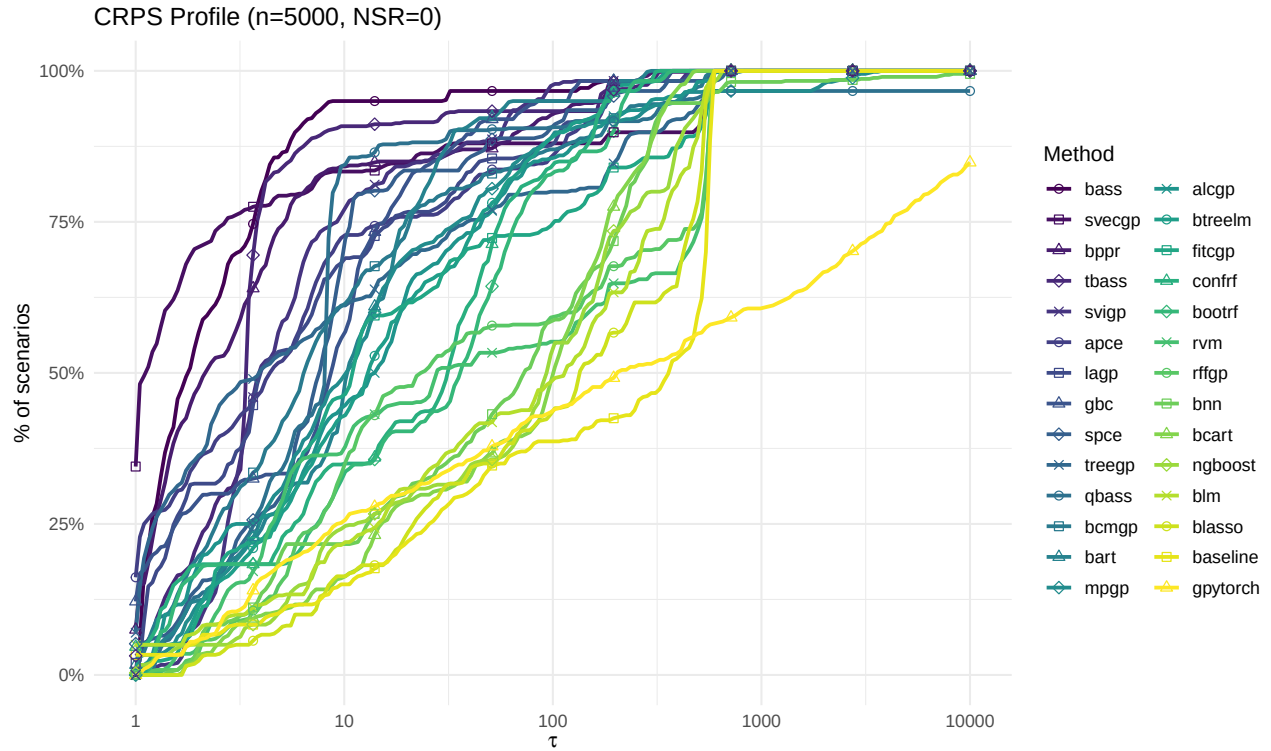
SM 5.2.2 CRPS Performance Profiles (testfuncs only)

```
n_train_vec <- c(500, 500, 1000, 1000, 5000)
NSR_vec      <- c(0, 0.1, 0, 0.1, 0)
for(i in 1:5){
  title <- paste0("CRPS Profile (n=", n_train_vec[i], ", NSR=", NSR_vec[i], ")")
  tmp <- filter_sim_study(duq_testfuncs,
                          n_train = n_train_vec[i], NSR = NSR_vec[i])
  p <- perfprofile_sim_study(tmp, metric="CRPS",
                             relative_epsilon = 0.001, tau_lim=c(1, 10000),
                             title=title, color_scale = "viridis")

  # Adjust sizing
  p <- p + ggplot2::theme_minimal(base_size=12)
  plot(p)
}
```







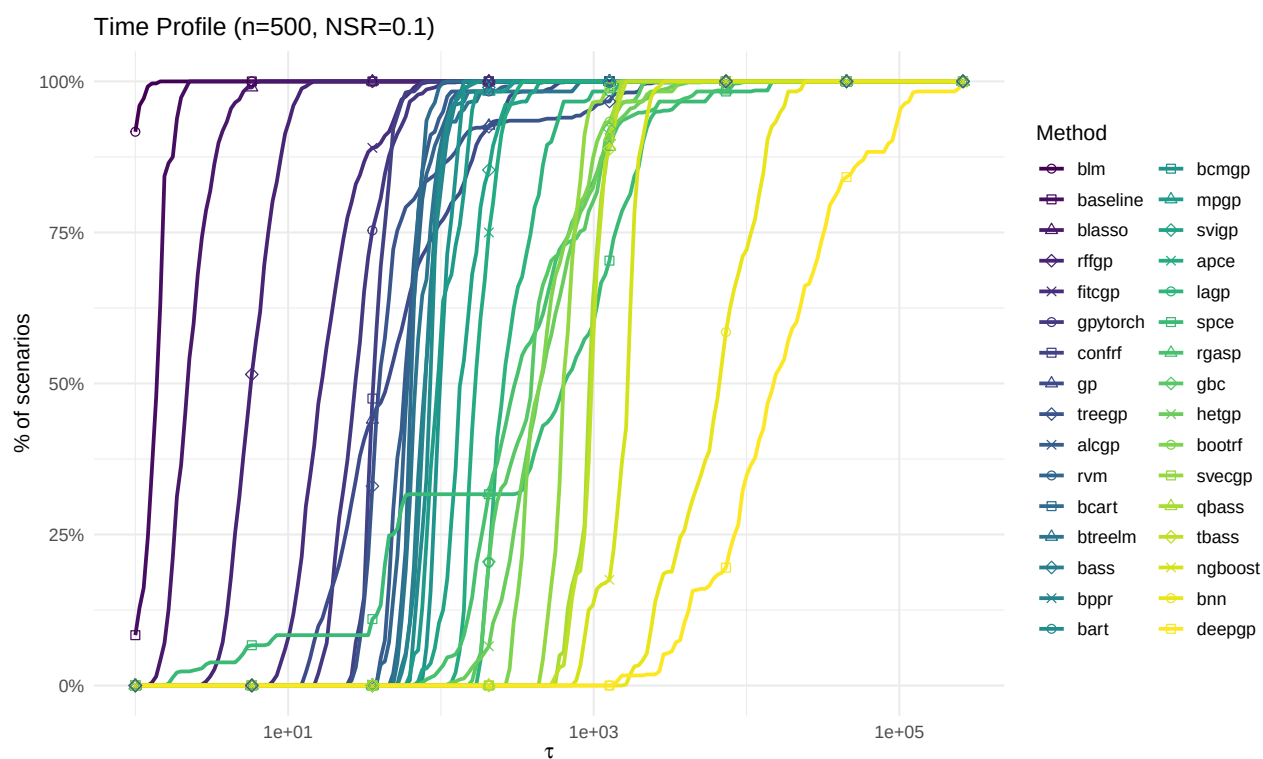
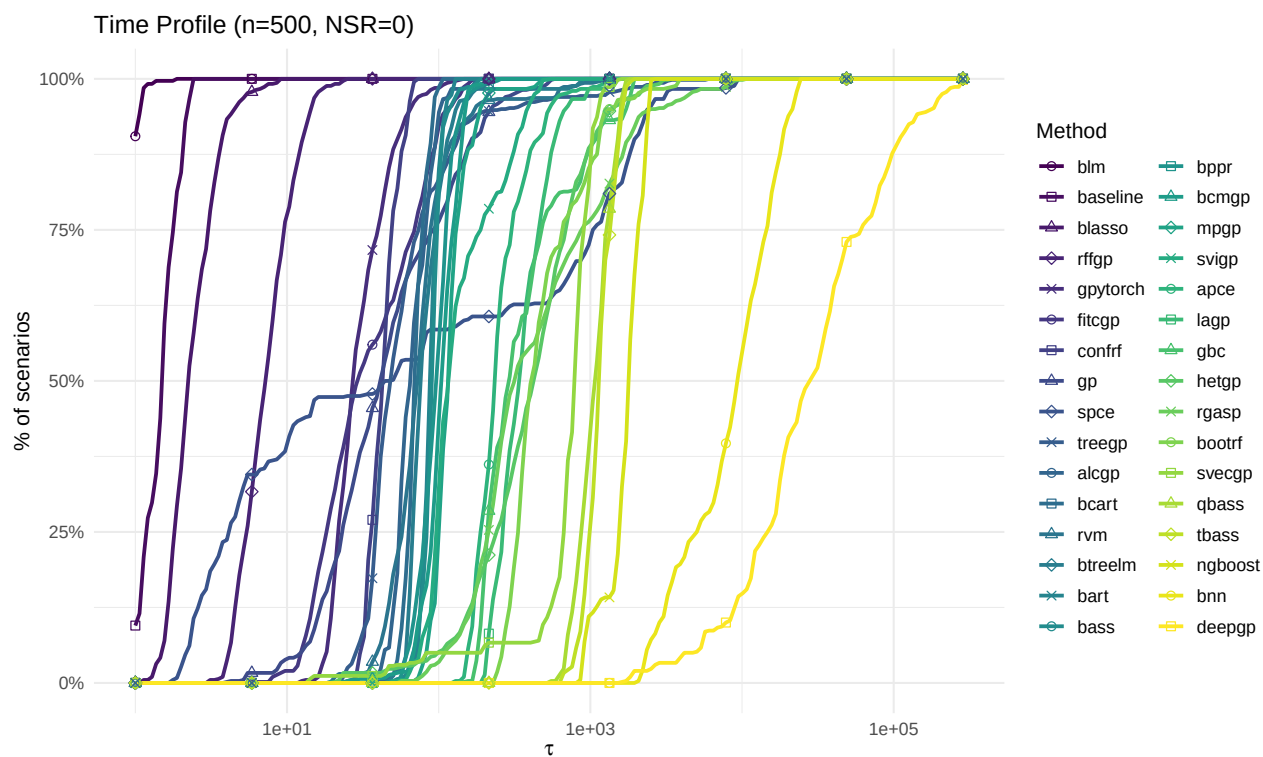
SM 5.3 Computation Time

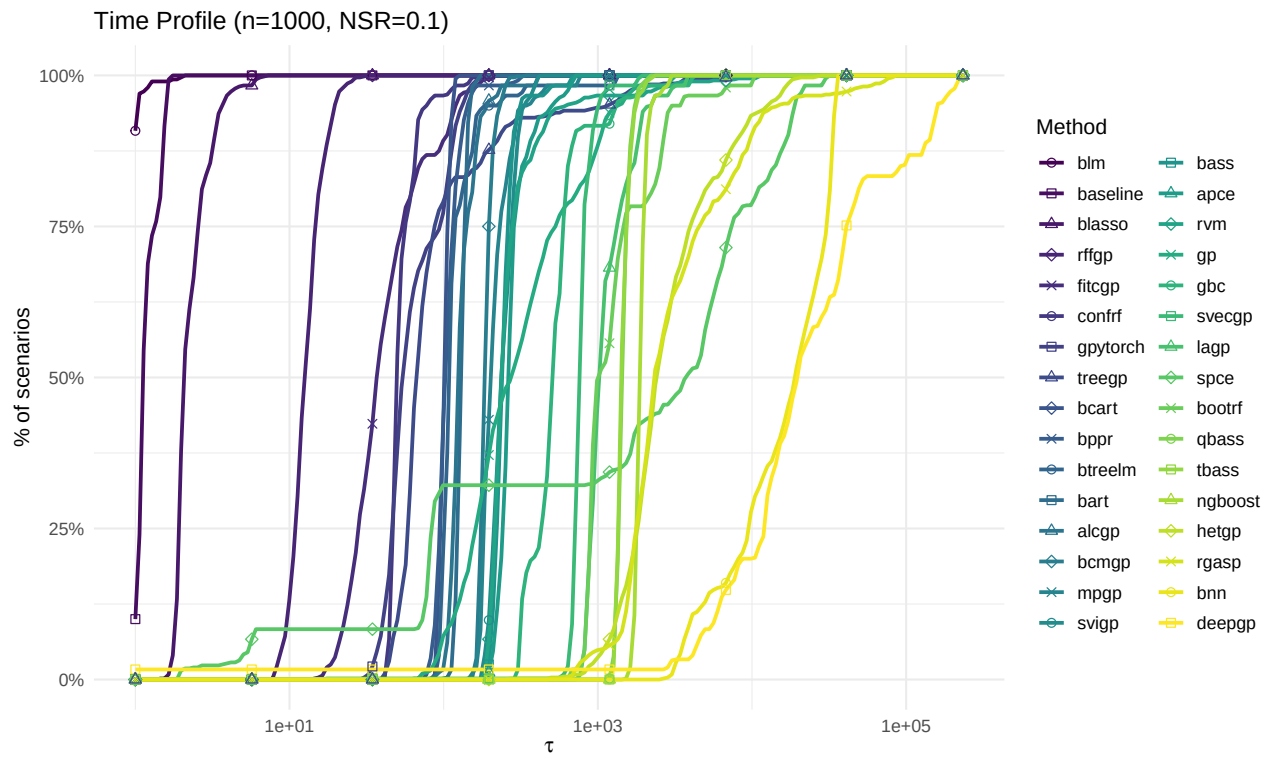
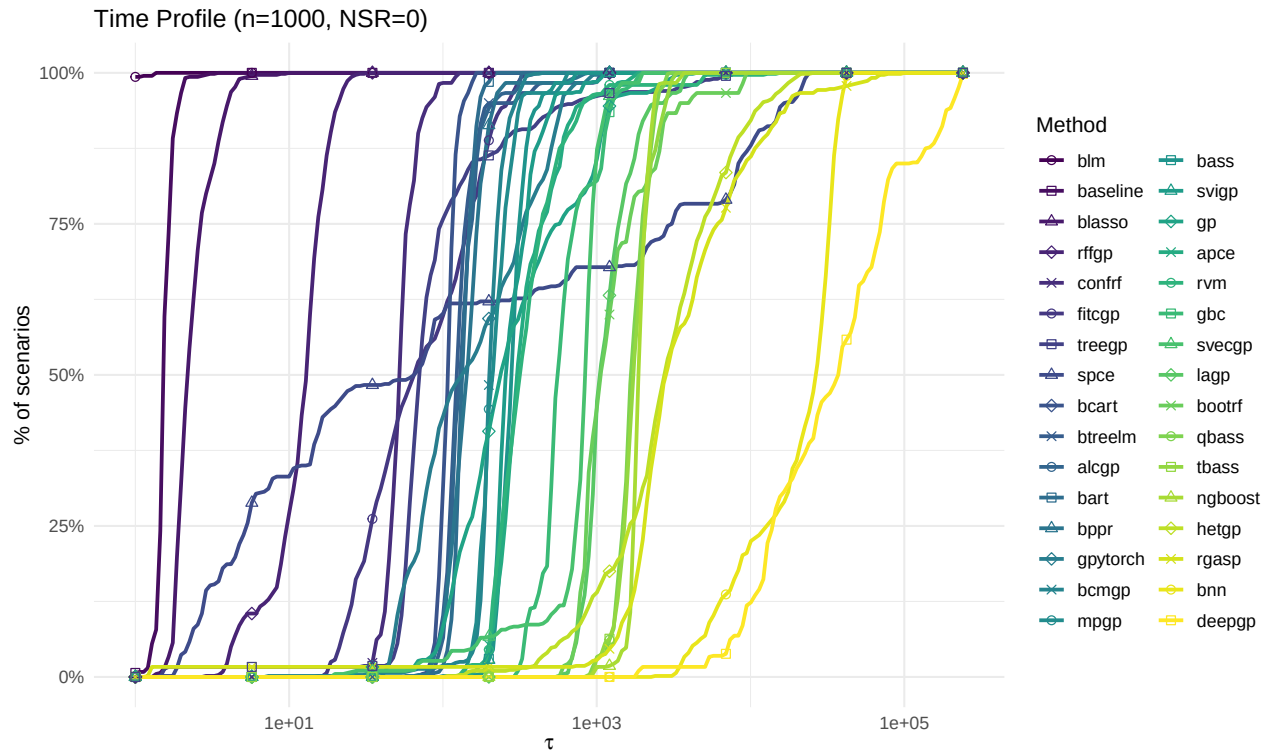
The `duqling` package usually times both training and prediction splits, but for this paper we report just the total computation time, because several of the surrogate modeling implementations cannot be split easily.

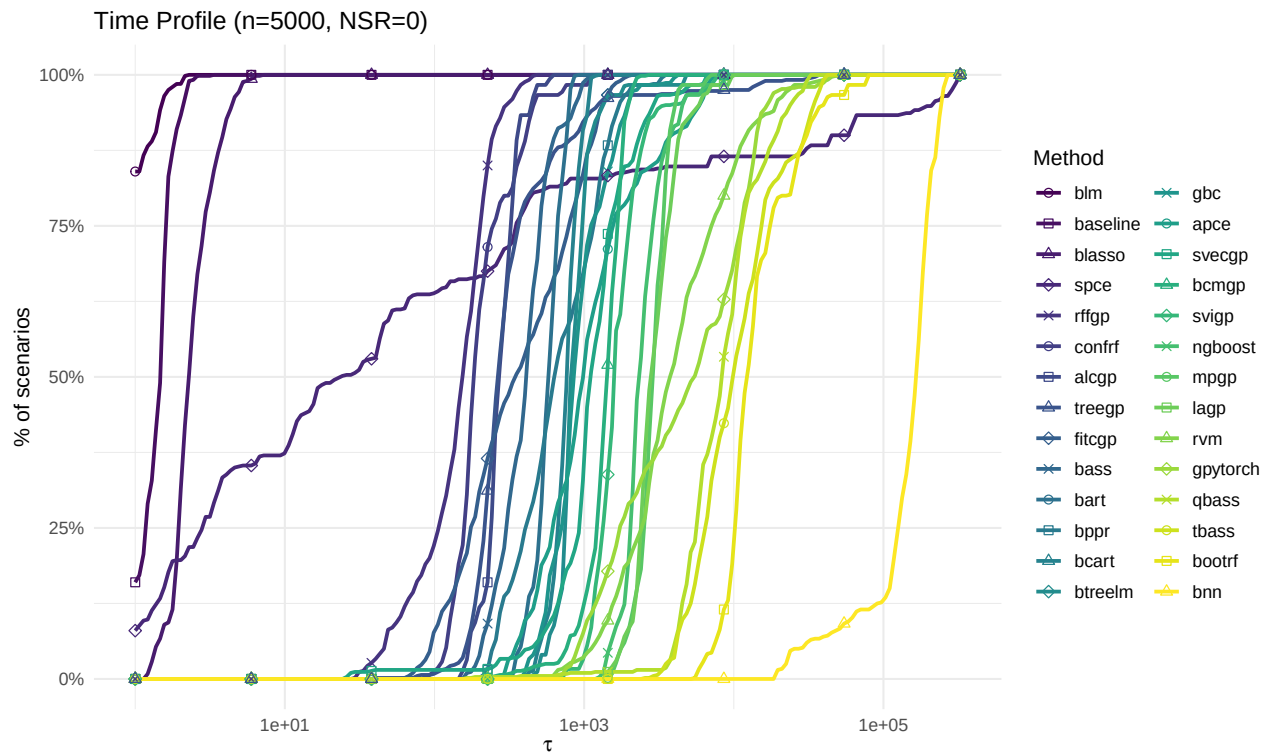
SM 5.3.1 Time Performance Profiles

```
# Synthetic test functions (5 cases)
n_train_vec <- c(500, 500, 1000, 1000, 5000)
NSR_vec     <- c(0, 0.1, 0, 0.1, 0)

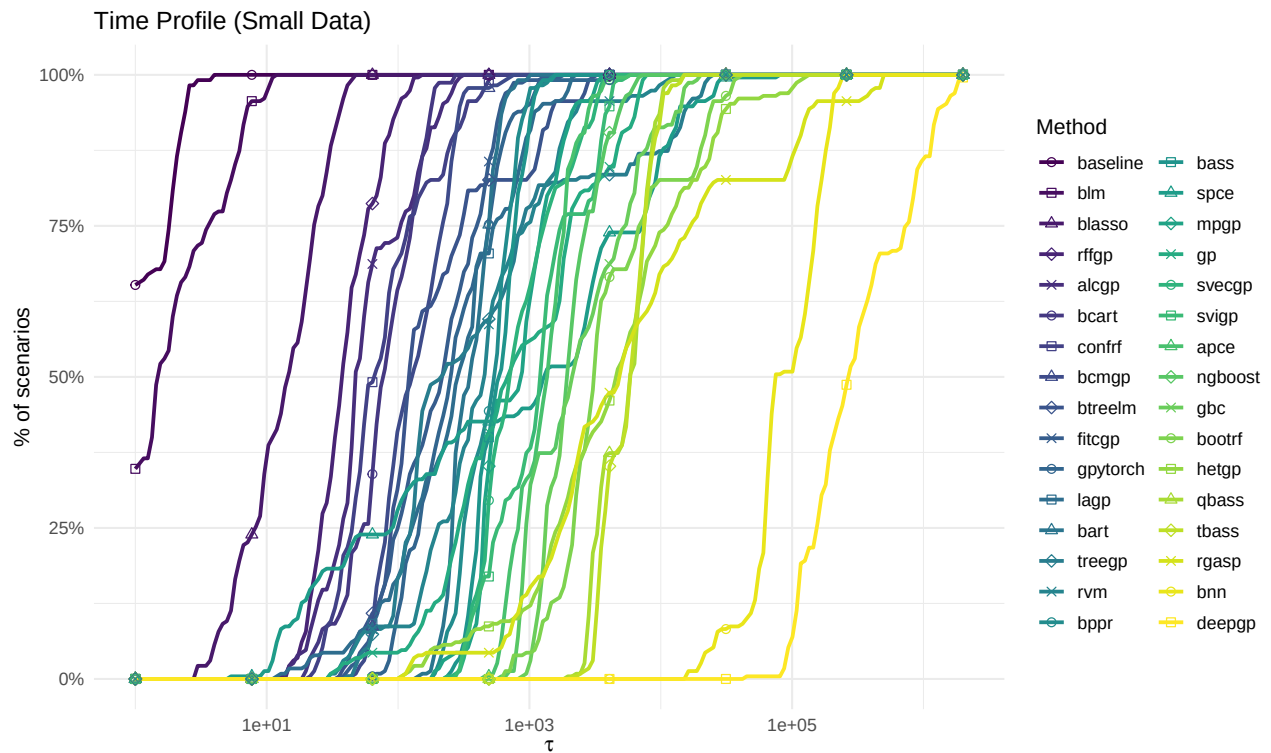
for(i in 1:5){
  title <- paste0("Time Profile (n=", n_train_vec[i], ", NSR=", NSR_vec[i], ")")
  tmp <- filter_sim_study(duq_testfuncs,
                          n_train = n_train_vec[i], NSR = NSR_vec[i])
  p <- perfprofile_sim_study(tmp, metric="time",
                             title=title, color_scale = "viridis")
  p <- p + ggplot2::theme_minimal(base_size=12)
  plot(p)
}
```



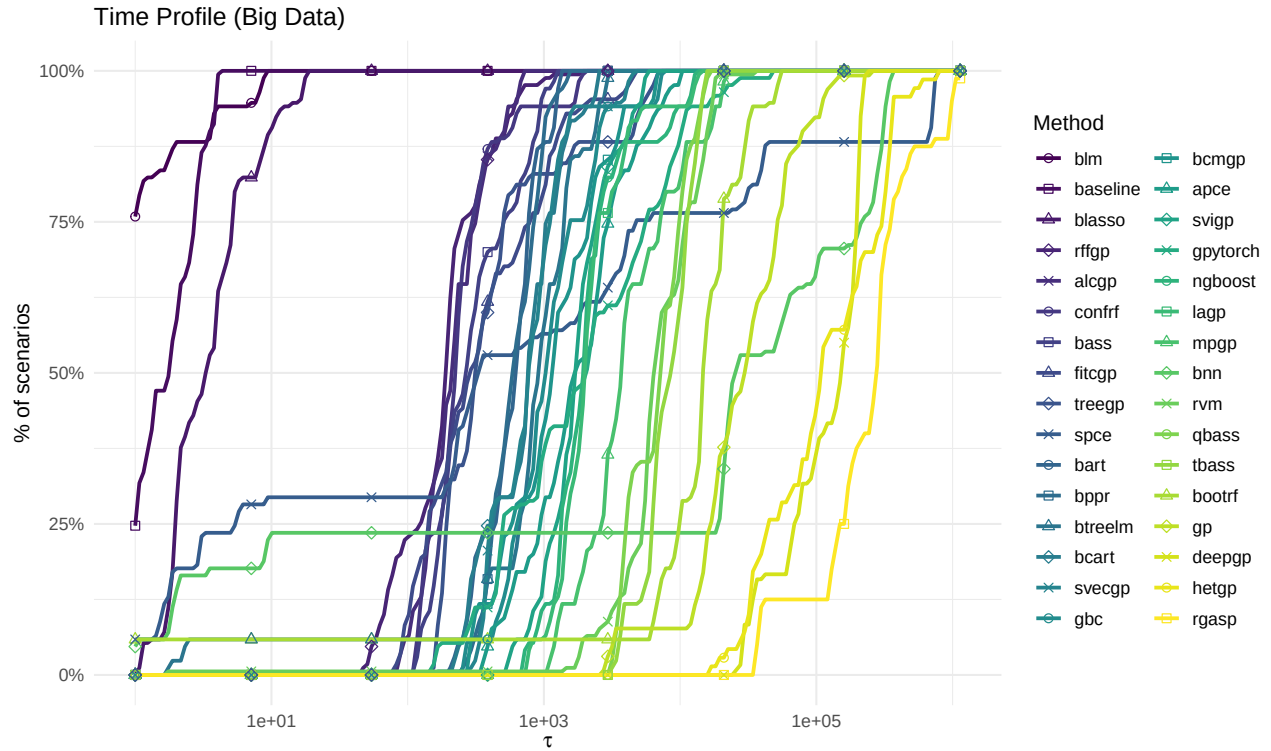




```
# Small data
title <- "Time Profile (Small Data)"
tmp <- filter_sim_study(duq_realddata, expr = n_train <= 1500)
p <- perfprofile_sim_study(tmp, metric="time",
                           title=title, color_scale = "viridis")
p <- p + ggplot2::theme_minimal(base_size=12)
plot(p)
```



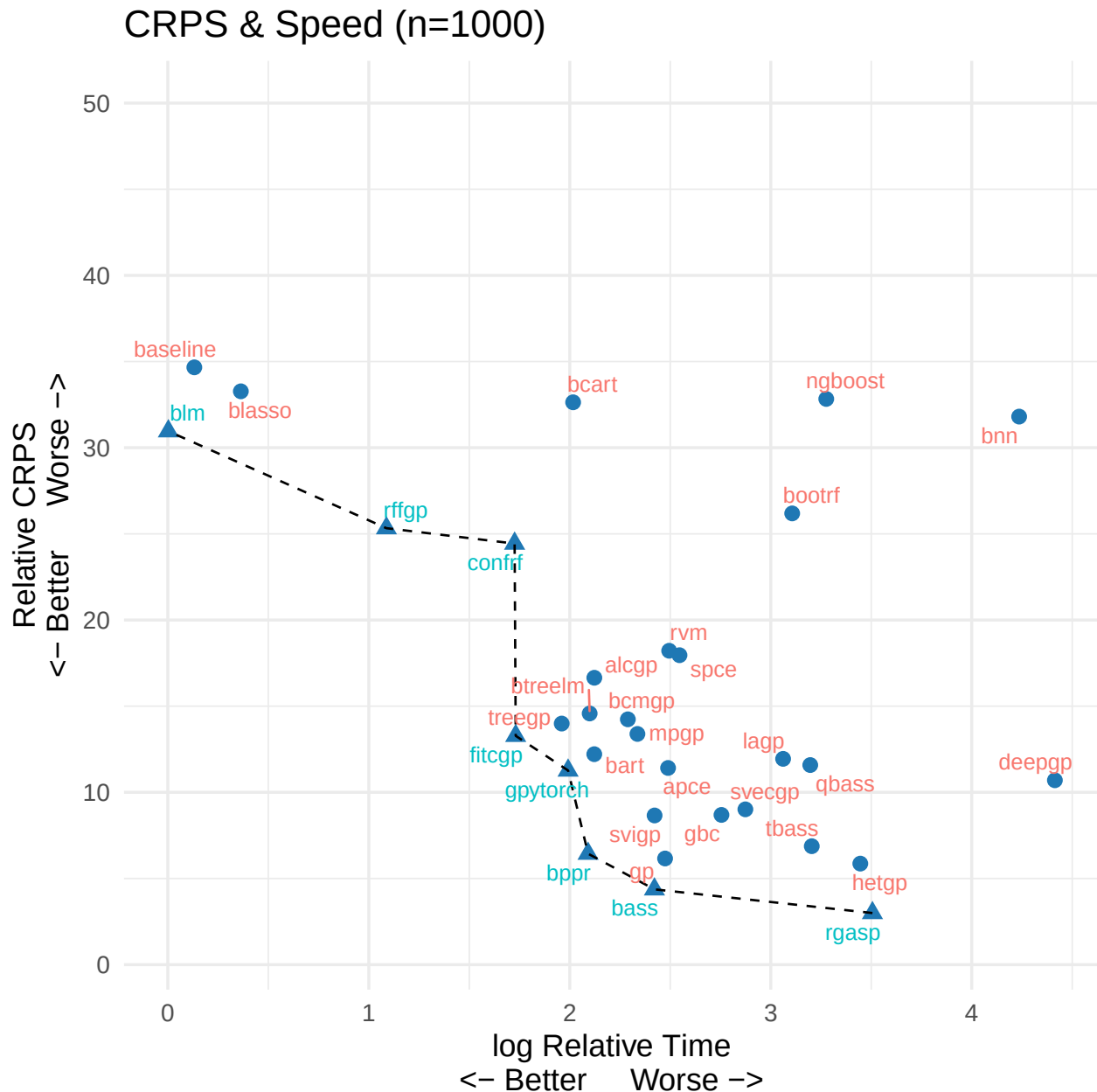
```
# Large data
title <- "Time Profile (Big Data)"
tmp <- filter_sim_study(duq_realddata, expr = n_train > 1500)
p <- perfprofile_sim_study(tmp, metric="time",
                           title=title, color_scale = "viridis")
p <- p + ggplot2::theme_minimal(base_size=12)
plot(p)
```



SM 5.3.2 Time Paretoplots

Finally, we provide an example of how the paretoplots in the main paper were created (with an internal call to `duqling::relativize_sim_study`).

```
# This only filters on sample size, not NSR
tmp <- filter_sim_study(duq_testfuncs, n_train=1000)
paretoplot_sim_study(tmp, metric=c("CRPS_rel", "time_rel_log"),
  epsilon=c(0.001, 0), upper_bound=c(100, Inf),
  show_legend = FALSE, title="CRPS & Speed (n=1000)",
  ylim=c(1, 50))
```



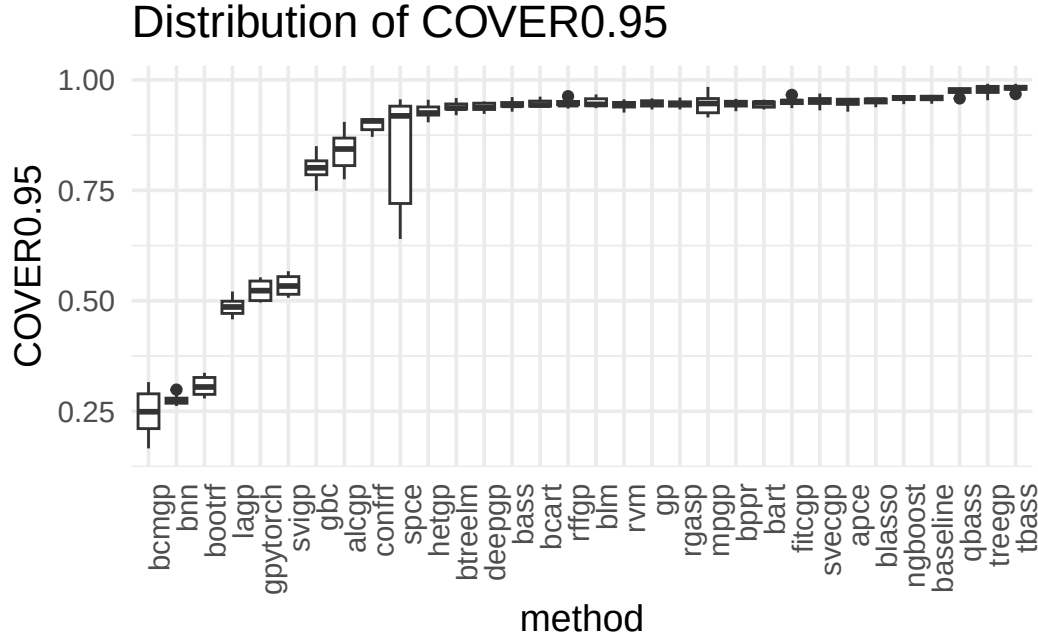
SM 5.4 Other Metrics

The analyses presented in the main paper focus primarily on CRPS, because it is a proper scoring rule for predictions with uncertainty, but the `duqling` framework is flexible and supports a wide range of alternative metrics. In many uncertainty quantification applications, practitioners are interested in properties of predictive intervals, such as coverage or interval width, in addition to overall predictive accuracy.

Coverage is one of the most commonly reported diagnostics, but it is not a proper scoring rule and does not account for the width of the interval.

```
tmp <- filter_sim_study(duq_testfuncs,
  id="borehole",
```

```
n_train=1000, NSR=0.1)
boxplots_sim_study(tmp, metric="COVER0.95")
```



For a given confidence level, mean interval scores (MIS) are a proper scoring rule that factor in both accuracy and precision.

$$\text{MIS}_{1-\alpha} = \frac{1}{n_{\text{test}}} \sum_{i=1}^{n_{\text{test}}} \left[(U_i - L_i) + \frac{2}{\alpha} (L_i - y_{\text{test},i}) \mathbf{1}\{y_{\text{test},i} < L_i\} + \frac{2}{\alpha} (y_{\text{test},i} - U_i) \mathbf{1}\{y_{\text{test},i} > U_i\} \right],$$

while the current version of **duqling** does compute these scores, they were omitted from the original simulation study for simplicity.

One simple metric due to [32] that can be recovered, is the integrated absolute error across confidence levels $1 - \alpha$, defined as

$$\text{IAE-}\alpha = \int_0^1 |C_\alpha - \alpha| d\alpha,$$

where C_α is the empirical coverage for confidence level α . To estimate this accurately in **duqling**, we should run with a long list of confidence levels using the `conf_level` argument in `run_sim_study()` and `run_sim_study_data()`. But it can be interesting to look at the estimate obtained using the confidence levels run by the default settings. In this case, we approximate the integral by a simple discrete average over the confidence levels used in the simulation, i.e., by averaging the absolute differences between empirical and nominal coverage at each level.

$$\hat{\text{IAE}} = \frac{1}{K} \sum_{k=1}^K |\hat{C}_{\alpha_k} - \alpha_k|.$$

We can create this new column using the package's `mutate_sim_study()` function.

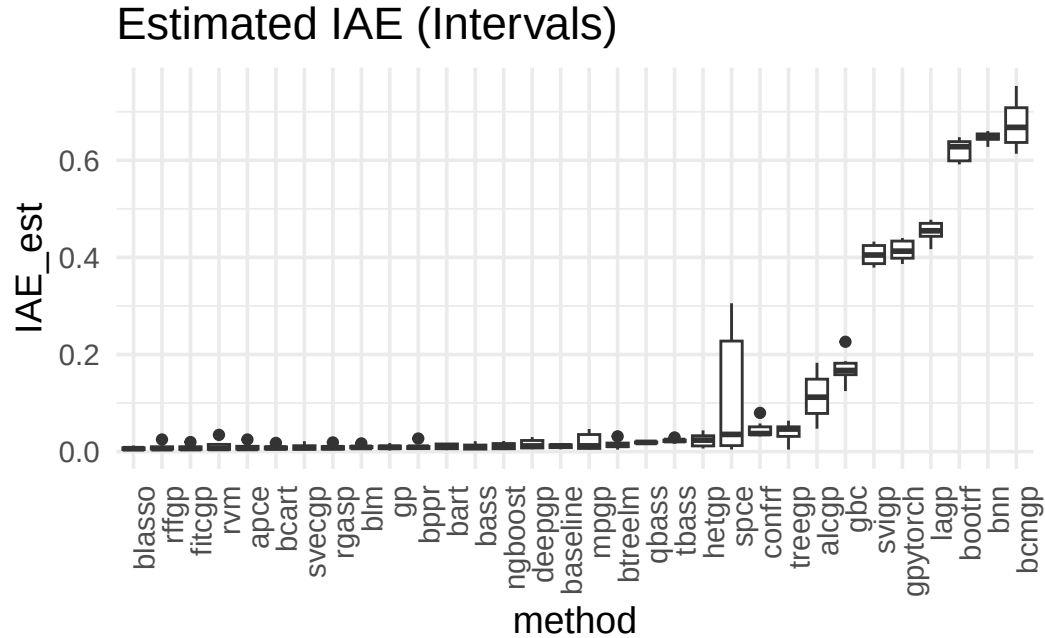
```
tmp_IAE <- mutate_sim_study(tmp,
  IAE_est = 0.25 * abs(COVER0.8 - 0.8) +
            0.25 * abs(COVER0.9 - 0.9) +
```

```

0.25 * abs(COVER0.95 - 0.95) +
0.25 * abs(COVER0.99 - 0.99))

boxplots_sim_study(tmp_IAE, metric = "IAE_est",
  title = "Estimated IAE (Intervals)")

```



These results indicate the shortcoming of IAE: they are an excellent tool for assessing how well calibrated a method is, but they fail to take precision of the predictive interval into account. Thus, the baseline predictor will often be reasonably well calibrated when the marginal output of the computer model is even remotely bell-shaped.

References

- [1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to algorithms*. MIT press, 2022.
- [2] D. Francom, B. Sansó, A. Kupresanin, and G. Johannesson, “Sensitivity analysis and emulation for functional data using bayesian adaptive splines,” *Statistica Sinica*, pp. 791–816, 2018.
- [3] T. Edmunds *et al.*, “The value of storage and demand response for renewable integration,” Lawrence Livermore National Laboratory (LLNL), Livermore, CA (United States), 2013.
- [4] E. Volkova, B. Iooss, and F. Van Dorpe, “Global sensitivity analysis for a numerical model of radionuclide migration from the RRC ‘kurchatov institute’ radwaste disposal site,” *Stochastic Environmental Research and Risk Assessment*, vol. 22, no. 1, pp. 17–31, 2008.
- [5] N. Klein, J. Colgan, E. Judge, K. Bhat, K. Myers, and E. Lawrence, “ATOMIC simulations and experimental data for basalt-like compounds.” Zenodo, Oct. 2020. doi: [10.25583/1670467](https://doi.org/10.25583/1670467).
- [6] K. N. Rumsey, Z. K. Hardy, C. Ahrens, and S. Vander Wiel, “Co-active subspace methods for the joint analysis of adjacent computer models,” *Technometrics*, vol. 67, no. 1, pp. 133–146, 2025.
- [7] IMPALA Team, *GitHub repository*. <https://github.com/lanl/impala>; GitHub, 2025.

- [8] D. L. Preston, D. L. Tonks, and D. C. Wallace, “Model of plastic deformation for extreme loading conditions,” *Journal of applied physics*, vol. 93, no. 1, pp. 211–220, 2003.
- [9] D. C. Francom *et al.*, “Simulation and emulation of x-ray diffraction from dynamic compression experiments,” *Journal of Dynamic Behavior of Materials*, vol. 7, no. LA-UR-19-31689, 2020.
- [10] A. Biswas *et al.*, “ASC/PEM/materials L3 milestone briefing: A briefing on impala and the savanna efforts in support of FY22/FY23 SCDS L1 milestones,” Los Alamos National Laboratory (LANL), 2021.
- [11] G. R. Johnson, “A constitutive model and data for metals subjected to large strains, high strain rates and high temperatures,” in *Proceedings of the 7th international symposium on ballistics, am. Def. Prep. Org.(ADPA), netherlands, 1983*, 1983, pp. 541–547.
- [12] D. J. Walters *et al.*, “Bayesian calibration of strength parameters using hydrocode simulations of symmetric impact shock experiments of al-5083,” *Journal of Applied Physics*, vol. 124, no. 20, 2018.
- [13] A. Hlobilová, S. Marelli, and B. Sudret, “Benchmark case datasets - two-dimensional heat diffusion model.” Zenodo, 2024. doi: [10.5281/zenodo.12701147](https://doi.org/10.5281/zenodo.12701147).
- [14] C. J. Smith *et al.*, “FAIR v1. 3: a simple emissions-based impulse response and carbon cycle model,” *Geoscientific Model Development*, vol. 11, no. 6, pp. 2273–2297, 2018.
- [15] S. Sjue, K. Hickmann, and N. Nguyen, “Simple and sophisticated models of taylor’s cylinder impact,” in *AIP conference proceedings*, AIP Publishing LLC, 2023, p. 370006.
- [16] K. N. Rumsey, D. Francom, and A. Shen, “Generalized bayesian MARS: Tools for stochastic computer model emulation,” *SIAM/ASA Journal on Uncertainty Quantification*, vol. 12, no. 2, pp. 646–666, 2024.
- [17] K. Sargsyan, C. Safta, H. N. Najm, B. Debusschere, D. Ricciuto, and P. Thornton, “Sparse polynomial chaos surrogate for ACME land model via iterative bayesian compressive sensing.” Sandia National Laboratories (SNL-CA), Livermore, CA (United States), 2015.
- [18] G. Hutchings, B. Sansó, J. Gattiker, D. Francom, and D. Pasqualini, “Comparing emulation methods for a high-resolution storm surge model,” *Environmetrics*, vol. 34, no. 3, p. e2796, 2023.
- [19] J. L. Brown and L. B. Hund, “Estimating material properties under extreme conditions by using bayesian model calibration with functional outputs,” *Journal of the Royal Statistical Society Series C: Applied Statistics*, vol. 67, no. 4, pp. 1023–1045, 2018.
- [20] M. Grosskopf *et al.*, “In-situ spatial inference on climate simulations with sparse gaussian processes,” in *ISAV’21: In situ infrastructures for enabling extreme-scale analysis and visualization*, 2021, pp. 31–36.
- [21] D. C. Francom, S. A. Vander Wiel, and B. P. Weaver, “Nuclear data dimension reduction,” Los Alamos National Lab.(LANL), Los Alamos, NM (United States), 2019.
- [22] J. A. Gaffney *et al.*, “The JAG inertial confinement fusion simulation dataset for multi-modal scientific deep learning,” *Lawrence Livermore National Laboratory (LLNL) Open Data Initiative. UC San Diego Library Digital Collections*, vol. 3, 2020.
- [23] D. Francom, B. Sansó, V. Bulaevskaya, D. Lucas, and M. Simpson, “Inferring atmospheric release characteristics in a large computer experiment using bayesian adaptive splines,” *Journal of the American Statistical Association*, vol. 114, no. 528, pp. 1450–1465, 2019.
- [24] M. J. Brown, A. Gowardhan, M. Nelson, M. Williams, and E. R. Paradyjak, “Evaluation of the QUIC wind and dispersion models using the joint urban 2003 field experiment dataset,” in *AMS 8th symp. Urban env, phoenix, USA*, 2009, pp. 10–16.

- [25] A. Fisher, C. Rudin, and F. Dominici, “All models are wrong, but many are useful: Learning a variable’s importance by studying an entire class of prediction models simultaneously,” *Journal of Machine Learning Research*, vol. 20, no. 177, pp. 1–81, 2019.
- [26] R. Xin, C. Zhong, Z. Chen, T. Takagi, M. Seltzer, and C. Rudin, “Exploring the whole rashomon set of sparse decision trees,” *Advances in neural information processing systems*, vol. 35, pp. 14071–14084, 2022.
- [27] C. Rudin *et al.*, “Amazing things come from having many good models,” *arXiv preprint arXiv:2407.04846*, 2024.
- [28] A. Visokay, E. Hock, K. Hoffman, and T. H. McCormick, “When you come to a fork in the road, take it: The rashomon effect for social scientists,” 2025.
- [29] L. Breiman, “Statistical modeling: The two cultures (with comments and a rejoinder by the author),” *Statistical science*, vol. 16, no. 3, pp. 199–231, 2001.
- [30] E. D. Dolan and J. J. Moré, “Benchmarking optimization software with performance profiles,” *Mathematical programming*, vol. 91, no. 2, pp. 201–213, 2002.
- [31] J. J. Moré and S. M. Wild, “Benchmarking derivative-free optimization algorithms,” *SIAM Journal on Optimization*, vol. 20, no. 1, pp. 172–191, 2009.
- [32] A. Marrel and B. Iooss, “Probabilistic surrogate modeling by gaussian process: A review on recent insights in estimation and validation,” *Reliability Engineering & System Safety*, vol. 247, p. 110094, 2024.